Correlation-Aware Machine Learning Task Scheduling for Power Efficiency and GPU Provisioning in AI Data Centers

A Thesis

Presented in Partial Fulfillment of the Requirements for the Degree Master of Science in the Graduate School of The Ohio State University

By

Srinivasan Subramaniyan, B.Tech

Graduate Program in Electrical and Computer Engineering

The Ohio State University

2026

Master's Examination Committee:

Dr. Xiaorui Wang, Advisor

Dr. Marco Brocanelli

© Copyright by

Srinivasan Subramaniyan

2026

Abstract

The rapid growth of Machine Learning as a Service (MLaaS) has driven the large-scale deployment of GPU data centers to support increasingly diverse and compute-intensive AI workloads. As these infrastructures expand, improving their efficiency has become a critical challenge for data center operators. In particular, operators must address two key questions: how to provision an appropriate number of GPUs to meet workload demands while minimizing capital expenditure (CapEx), and how to efficiently operate deployed GPU clusters to reduce power consumption and operational expenditure (OpEx).

This thesis addresses these challenges through correlation-aware GPU scheduling. First, we focus on GPU provisioning and propose `CorrGPU`, a correlation-aware scheduling algorithm that minimizes the number of GPUs needed to process ML training workloads while meeting their desired deadlines. `CorrGPU` is motivated by a key observation from real-world traces that GPU utilization for ML tasks varies over time and often exhibits weak or negative correlations. By exploiting these correlations, `CorrGPU` enables efficient task consolidation, reducing GPU provisioning requirements and lowering CapEx.

Second, we address runtime efficiency and propose `CATS`, a correlation-aware scheduling framework for reducing GPU power consumption in deployed data centers. `CATS` leverages complementary utilization patterns among tasks to consolidate workloads

onto fewer GPUs, allowing idle GPUs to be placed into low-power states. In addition, `CATS` integrates dynamic GPU frequency scaling to further reduce power consumption while maintaining acceptable job completion times. To achieve this, we formulate the scheduling problem as a constrained optimization problem and design an efficient heuristic algorithm for scalable deployment.

Extensive evaluations using both real-world traces and hardware testbeds demonstrate that `CorrGPU` reduces GPU provisioning cost by up to 20.88%, while `CATS` achieves up to 24.7% reduction in power consumption with minimal performance impact. Together, these contributions provide a holistic solution to reduce both cost and power consumption in modern GPU data centers.

This thesis is dedicated to my mother, Seethalakshmi; my father, T.S. Subramanian; and my sister, Janani.

Acknowledgments

I would like to express my deepest gratitude to everyone who has supported me throughout the journey of this MS thesis. Without their guidance, encouragement, and assistance, this work would not have been possible.

I would like to begin by expressing my profound gratitude to my advisor, Dr. Xiaorui Wang, for his unwavering support, insightful guidance, and the many opportunities he provided that have helped shape me as a researcher. I am also deeply grateful to Dr. Marco Brocanelli for his valuable guidance and for graciously serving on my thesis committee.

I would like to express my heartfelt gratitude to my mother, Seethalakshmi, my father, T. S. Subramanian, my sister, Janani, and my grandmother, Gomathy. Their unconditional love and encouragement inspire me to improve myself every day. I would also like to extend my special thanks to my uncle, Ganesh Srinivasan, and my cousins, Varshini, Abhinav, and Vidhya, for always being there for me. Additionally, I am deeply grateful to my relatives in the U.S., Ganesh Ramaswamy and his family, and Mani Ramaswamy and his family, for their valuable insights and guidance, and for always making me feel at home.

I would like to thank my labmates for their camaraderie and collaboration: Guoyu Chen, Yuan Ma, Simon Wu, Kailun Lin, Ze Li, Rudra Joshi, Michael Zhong, Akshar

Chavan, Syeda Tanjila Atik, Keegan Freyhoff, Blake Morse, Aadi Pathak, Yuanyang Cheng, Ze Pi, Xiaochen Fan, Shang-Ru Yao, Sajjad Akherati, and Jiaxuan Cai.

Finally, I would like to express my gratitude to all my friends at The Ohio State University and during my internships. While there are too many of you to name individually, please know that I cherish each of those friendships deeply. Your presence made this journey far more meaningful and memorable.

Vita

2015-2019	B.Tech Electronics and Communication Engineering, Amrita Vishwa Vidyapeetham University
2021-2026	MS in Electrical and Computer Engineering at The Ohio State University.
2022-present	Graduate Student Researcher, The Ohio State University.

Publications

Research Publications

S. Subramaniyan and X. Wang "Exploiting ML Task Correlation in the Minimization of Capital Expense for GPU Data Centers". *Proceedings of the IEEE International Performance, Computing, and Communications Conference (IPCCC)*, Austin, TX, USA, 2025, pp. 1–8. [IPCCC 2025]

S. Subramaniyan and X. Wang "CATS: Correlation-aware Task Scheduling for GPU Power Optimization in AI Data Centers". *Proceedings of the International Conference on Supercomputing (ICS)*, Belfast, United Kingdom, July 2026. [ICS 2026]

Fields of Study

Major Field: Electrical and Computer Engineering

Table of Contents

Page

Abstract ii

Dedication iv

Acknowledgments v

Vita vii

List of Tables x

List of Figures xi

1. Introduction 1

    1.1 GPU Provisioning for ML Workloads 1
    1.2 GPU Power Optimization in Data Centers 3
    1.3 Contribution 5

2. Exploiting ML Task Correlation in the Minimization of Capital Expense for GPU Data Centers 8

    2.1 Introduction 8
    2.2 Related Work 10
    2.3 Problem Formulation 12
    2.4 Design of `CorrGPU` 15
        2.4.1 Correlation Analysis of Machine Learning Workloads 15
        2.4.2 Example 18
        2.4.3 Framework Design 22
        2.4.4 Complexity Analysis 25
        2.4.5 Practical Considerations 26
    2.5 Experimental Setup 27

2.6 Hardware Evaluation 28
2.7 Simulation Evaluation 29
2.8 Conclusion 32

3. CATS 34

3.1 Introduction 34
3.2 Related Work 36
3.3 Problem Formulation 38
3.3.1 Modeling 38
3.3.2 Problem Formulation 42
3.4 Design of `CATS` 44
3.4.1 GPU Utilization 45
3.4.2 Utilization Correlation Analysis 46
3.4.3 Consolidation Analysis 49
3.4.4 `CATS` Framework 51
3.4.5 Correlation-Aware Consolidation Algorithm 54
3.4.6 Discussion 60
3.5 Experimental Setup 61
3.6 Hardware Evaluation 63
3.7 Simulation Evaluation 66
3.8 Conclusion and Future Work 71

4. Conclusion and Future Work 72

Bibliography 73

List of Tables

Table **Page**

2.1 Computation Time for Varying Numbers of Tasks 15

2.2 Number of Inequalities for Different Numbers of Tasks 15

3.1 Overhead for Correlation Computation 49

3.2 GPU Configuration in the Hardware Testbed 62

3.3 Task start time, GPU utilization (%), and memory consumption (GB). 64

List of Figures

Figure **Page**

2.1 (a) Scatter plot showing the distribution of pairwise correlation coefficients. (b) Cumulative Distribution Function (CDF) plot depicting the overall distribution pattern of the pairwise correlation coefficients. 17

2.2 (a) GPU utilization and (b) CapEx comparison in the motivational example. `CorrGPU` yields the lowest CapEx among all baselines. . . . 18

2.3 `CorrGPU` uses only one GPU and decreases the *CapEx* for the motivational example. 19

2.4 Hardware evaluation using the PAI trace. (a) `CorrGPU`'s *CTD* is smaller than that of *Optimal* and remains within the due time. (b) `CorrGPU` uses fewer GPUs compared to *Harmony* and *First*, resulting in a lower *CapEx*. 30

2.5 `CorrGPU` uses fewer GPUs compared to the baselines for the large-scale simulation using the PAI trace. 31

2.6 (a) CorrGPU exhibits a slightly higher Cumulative Task Duration (CTD) than Harmony (0.89) and First (0.96), but remains within the acceptable due times. (b) CorrGPU achieves significantly lower *CapEx* than both baselines by reducing the total number of unique GPUs provisioned. 32

3.1 Measured and predicted GPU power consumption and job completion time (JCT) under different SM frequencies: (a) power consumption and (b) JCT. 41

3.2 (a) The CDF of the correlation coefficient for the 600 tasks chosen from the Philly trace. (b)The CDF of DRAM usage for the 600 selected tasks from the GPUs in the PAI trace. 48

3.3 Architecture of the proposed CATS framework. The framework comprises correlation analysis, a scheduler, and a power-state manager that performs periodic sleep and frequency adaptation. 51

3.4 The two tasks are negatively correlated, and the aggregated utilization is less than 100%. 52

3.5 An example of ML task consolidation by CATS. (a) shows the initial task assignment. (b) shows the aggregated task consolidation after applying CATS. We put the unused GPU (GPU 1), to sleep to reduce power consumption. 53

3.6 (a) The optimal solution has the smallest power consumption. (b) The normalized JCT is similar across all solutions, and the optimal solution remains within the SLA threshold. 65

3.7 Resource utilization, performance, and power consumption results for the PAI trace simulation. (a) CATS uses the fewest GPUs. (b) The normalized job completion time (JCT) of CATS remains within the SLA. (c) CATS uses the fewest servers. (d) CATS achieves the lowest average power consumption. 68

3.8 Resource utilization, performance, and power consumption results for the Philly trace simulation. (a) CATS uses the fewest GPUs. (b) The normalized job completion time (JCT) of CATS remains within the SLA. (c) CATS uses the fewest servers. (d) CATS achieves the lowest average power consumption. 69

3.9 (a) CATS achieves the lowest power consumption. (b) Normalized JCT for all the solutions. 70

Chapter 1: Introduction

Machine Learning as a Service (MLaaS) has become increasingly popular in recent years, offering an attractive alternative for users who cannot afford or maintain high-end GPUs [1]. MLaaS platforms support widely used frameworks such as TensorFlow [2], PyTorch [3], and Caffe [4], enabling applications such as image recognition [5], natural language processing [6], and transfer learning. The rapid growth of MLaaS has driven the construction of large-scale GPU data centers. As GPU clusters become increasingly central to modern AI infrastructure, improving their efficiency has become a critical challenge. In this thesis, we address GPU data center efficiency from two complementary perspectives. **Before deployment**, operators must determine the number of GPUs to provision to meet workload demands while minimizing capital expense (CapEx). **After deployment**, operators must manage GPU resources efficiently to reduce power consumption and operational expense (OpEx). Together, these two challenges define the lifecycle of efficient GPU data center management.

1.1 GPU Provisioning for ML Workloads

The rapidly growing demand for such MLaaS services has been driving the construction of new GPU data centers. However, an important question for many data center operators is: How many GPUs should be purchased for such a data center, so

that 1) all the ML training tasks directed to this data center can be serviced in a timely manner, and 2) the capital expense (CapEx) of the data center can be minimized by purchasing only the smallest number of GPUs needed for those tasks. Any over-provisioning of GPUs can lead to wasted computing resources and unnecessarily higher CapEx, whereas any under-provisioning can increase GPU resource contention among tasks during peak times, thereby delaying their completion.

A common practice among many data center operators is to analyze real-world traces from a similarly sized data center (e.g., Alibaba's PAI trace [7] or Microsoft's Philly trace [8]). Then, the number of GPUs required to run all tasks in a timely manner can be estimated with reasonable accuracy. This approach enables data center operators to make informed decisions about GPU provisioning and optimize resource allocation for efficient data center operations. Specifically, we aim to minimize the number of GPUs required to process all training tasks in the given ML traces, while meeting the desired task *due times*, which are user-defined deadlines for those tasks to be completed.

Many scheduling solutions have been previously proposed for ML training tasks [9–13]. While those solutions rely on various metrics for scheduling, GPU utilization (i.e., the percentage of time a GPU is actively involved in computation) is commonly used to schedule multiple tasks on the same GPU if their aggregated GPU utilization is lower than the GPU capacity (e.g., 100%), subject to other constraints like memory [7, 11,14]. However, utilization-based solutions typically assume that the GPU utilization of each task can be approximated by a constant value (e.g., peak or average utilization) and then use that constant in their scheduling. This oversimplified approach may not accurately capture the fluctuating nature of GPU utilization in ML training

workloads, as observed in empirical traces (e.g., [7] [8]), where GPU utilization for many tasks varies significantly over their duration. Hence, such an approximation may not yield optimal scheduling results. For example, if peak utilization is always assumed, operators may end up purchasing an unnecessarily large number of GPUs, leading to higher CapEx. If the average value is always used, when multiple tasks are scheduled on the same GPU and peak simultaneously, intensive GPU contention may cause them to violate their desired due times. This observation highlights the need for more accurate scheduling methods that capture the dynamic behavior of ML workloads.

While GPUs are widely recognized for their versatility across domains, including Parallel Computing [15, 16], Error-Correcting Codes [17–19], Computational biomechanics [20, 21], Computational mechanics [22, 23], Computational materials science [24, 25], Predictive maintenance [26] , robotics [27], and AR/VR computing [28, 29], this research focuses on the critical area of GPU scheduling in the cloud. Despite the diverse computational applications of these processors, optimizing their resource management and allocation within cloud environments remains the primary objective of this work.

1.2 GPU Power Optimization in Data Centers

The recent success of AI/ML development has resulted in a high demand for Graphics Processing Units (GPUs). As an arms race, major IT companies (e.g., OpenAI, Meta, Microsoft, Google) are all rushing to purchase/build more GPU chips to significantly enhance their AI/ML infrastructures [30]. Hence, it can be well anticipated that the number of GPUs used in the data centers of those Big Tech

companies will increase sharply in the next few years. Although GPUs are generally more energy-efficient than CPUs, a GPU can consume significantly more power than a CPU because of its higher density of memory and computing units that operate in parallel. For example, the power consumption of a high-end GPU used for ML Training (e.g., Nvidia H100) can be as high as 700 Watts [31]. Based on Nvidia's estimate, approximately 1.5 to 2 million H100 GPUs were sold in 2024. Thus, the total power consumption of just those H100 GPUs can reach 13.1 TWh annually [32], which, if compared to residential power consumption of large cities in the US, would rank as the fifth-largest residential power consumer and is higher than the entire city of Phoenix, AZ [33]. Therefore, it is crucial to optimize the power consumption of these massive GPU data centers used for AI/ML development.

Most existing GPU power optimization research relies on offline profiling of the target ML applications on a specific GPU to identify the most energy-efficient scheduling (e.g., [34, 35]). However, they may incur high profiling overheads and cannot adapt to online changes in program behaviors. Some studies aim to modify the internal mechanisms of specific ML models to improve energy efficiency [36], but these approaches may not apply to all ML applications. Online system-level GPU power optimization has been proposed for ML inference [14,37–41] and training tasks [42,43], respectively. However, they mainly depend on GPU frequency scaling to achieve the desired power-performance trade-offs. While frequency scaling can provide prompt GPU power management, it cannot eliminate GPU idle power consumption, which can be significant for many GPUs, and thus achieves only inferior power optimization results.

A well-known approach to minimizing idle power consumption in data centers is to consolidate workloads onto fewer computational resources, such as GPUs or CPUs, to optimize resource utilization [44] [45]. Then, unused computational resources can be turned off or put into sleep [44] [46], reducing the data center's power consumption. For example, task consolidation for data center server power optimization has been studied before to make servers more energy-proportional to their workloads [47, 48]. Likewise, the traffic flows in a data center network can also be consolidated to put unused network devices into sleep for energy savings [49]. For GPU data centers, recent research has proposed various methods for packing more ML tasks onto fewer GPUs [11–13] with the goal of increasing GPU utilization, because GPUs in production data centers are reported to be severely underutilized [50]. With such GPU-sharing or consolidation methods, unused GPUs can be powered down or placed in sleep mode to minimize their idle power consumption. Additionally, when no GPUs in a server are active, we can further reduce idle power consumption by putting unused servers to sleep [44] [46].

1.3 Contribution

In this thesis, we address GPU data center efficiency across both deployment and runtime phases, making two key contributions toward reducing capital expenditure (CapEx) and operational expenditure (OpEx).

First, to minimize the number of GPUs required in a data center, we propose `CorrGPU` [51] in Chapter 2. `CorrGPU` is a correlation-aware GPU scheduling algorithm that leverages utilization correlations among ML training tasks to reduce GPU provisioning requirements. This design is motivated by a key observation from real

ML traces: different tasks rarely reach their peak GPU utilization simultaneously. By analyzing and exploiting these correlations, `CorrGPU` enables more tasks to be co-scheduled on the same GPUs without violating job completion constraints. For example, tasks with negative utilization correlation can share a GPU since their peak demands do not overlap. Since modern data centers typically separate training and inference workloads to ensure low-latency inference [11, 52], `CorrGPU` focuses on training tasks, which have longer durations and provide sufficient data for correlation analysis. As a result, `CorrGPU` helps data center operators reduce the number of GPUs required, thereby lowering CapEx during both initial deployment and future scaling.

Second, to improve runtime power efficiency in existing GPU data centers, we propose `CATS` [53] in Chapter 3. `CATS` is a correlation-aware scheduling framework that reduces GPU power consumption by consolidating ML training tasks based on their utilization patterns. By scheduling tasks with complementary (e.g., negatively correlated) resource demands, `CATS` maximizes GPU utilization, allowing idle GPUs to be put into low-power states. Furthermore, `CATS` integrates dynamic GPU frequency scaling to reduce dynamic power consumption while maintaining acceptable job completion times (JCTs). Similar to `CorrGPU`, `CATS` focuses on training workloads due to their longer execution times and richer utilization patterns [52, 54, 55]. To achieve this, we first model the relationship between GPU frequency, power consumption, and task performance. We then formulate the consolidation problem as a constrained optimization problem, which is NP-hard, and derive an optimal solution. To ensure scalability, we design an efficient heuristic algorithm that performs correlation-aware

task consolidation with low computational overhead. Finally, we address practical deployment considerations, including scalability, utilization prediction, and integration with existing data center infrastructure.

Finally, we conclude the thesis and discuss potential directions for future work in Chapter 4.

Chapter 2: Exploiting ML Task Correlation in the Minimization of Capital Expense for GPU Data Centers

2.1 Introduction

As discussed in Chapter 1, a fundamental challenge in GPU data centers is determining how many GPUs should be provisioned to serve machine learning (ML) workloads within their desired due times while minimizing capital expense (CapEx). Over-provisioning leads to wasted resources and increased infrastructure cost, whereas under-provisioning results in resource contention and delayed task completion.

Existing approaches typically rely on GPU utilization to guide task scheduling and provisioning decisions. However, these approaches often assume that task utilization can be approximated using constant values such as peak or average utilization. As highlighted in Chapter 1, such simplifications fail to capture the dynamic, time-varying nature of GPU utilization in real-world ML workloads. Consequently, they can lead to inefficient provisioning decisions, either by overestimating resource requirements or by causing contention when tasks peak simultaneously.

To minimize the number of GPUs needed in a data center, we propose `CorrGPU`, a correlation-aware GPU scheduling algorithm that considers the utilization correlation among different ML training tasks to reduce needed GPUs. `CorrGPU` is designed

based on a key observation from the analysis of real ML traces that different tasks do not have their GPU utilization peak at exactly the same time. As a result, if the correlations among tasks are analyzed and accounted for in scheduling, more tasks can be scheduled on the same GPUs, without extending the training duration beyond the desired due time. For example, if two tasks have negative utilization correlation, they can be co-scheduled on the same GPU because they do not simultaneously demand for a large amount of GPU time. Because data centers often provision training and inference GPUs as separate clusters to ensure that inference is processed promptly without being affected by training [11, 52], `CorrGPU` focuses on scheduling training tasks, since inference tasks typically have very short durations, usually in milliseconds [11, 14, 54, 56, 57]. Therefore, we focus on training tasks with longer durations and provide multiple data points for correlation analysis. For a GPU data center to be constructed, `CorrGPU` can help operators purchase fewer GPUs, thereby minimizing CapEx. Even for an existing data center with pre-purchased GPUs, `CorrGPU` can help operators avoid purchasing additional GPUs as workloads increase due to the rapidly growing ML business, thereby reducing CapEx.

Specifically, this work makes the following contributions:

- By analyzing real-world ML traces, we observe that ML tasks usually have weak pairwise correlations and thus do not peak at the same time. Based on this observation, we propose scheduling ML tasks according to their correlations to minimize the number of required GPUs and lower CapEx.

- We formulate the problem of minimizing the number of GPUs needed in a data center as a constrained optimization problem. Due to its high complexity, we derive the optimal solution, which is impractical to use in a real data center due

to its computational overhead, but can serve as an upper bound for the design of heuristic algorithms.The objective function and the constraints are discussed in Section 2.3.

- We propose `CorrGPU`, a correlation-aware GPU scheduling algorithm that accounts for utilization correlations among tasks to minimize the number of GPUs required in a data center. With a much smaller overhead, the scheduling performance of `CorrGPU` is close to that of the optimal solution.

- We evaluate `CorrGPU` both on our hardware GPU testbed with Nvidia V100 GPUs and in trace-driven simulation. Our testbed results demonstrate `CorrGPU's` potential to reduce the number of required GPUs. Our large-scale simulation results also show that `CorrGPU` outperforms several state-of-the-art solutions by having 20.88% less CapEx.

The rest of this chapter is organized as follows. Section 2.2 reviews the related work. Section 2.3 formulates the optimization problem. Section 2.4 analyzes the correlations of popular machine learning workloads and presents the `CorrGPU` framework. Section 2.5 describes the experimental setup used for this chapter. Section 2.6 presents the hardware testbed results. Section 2.7 presents the simulation results. Section 2.8 concludes this Chapter.

2.2 Related Work

A large amount of research work has been carried out on studying scheduling training workloads in GPU data centers. These studies focus on efficient resource utilization, cost, and timing. The related work can be classified into two categories:

(1) Popular solutions used in the industry for scheduling training tasks in data centers and (2) Training scheduling techniques used in the literature.

Industrial solution. Alibaba [58] conducted a comprehensive study on a GPU cluster comprising 6000 GPUs dedicated solely to machine learning workloads. The investigation delved into load imbalances and low GPU resource utilization. Additionally, the study critically analyzed the limitations of elastic scheduling within data centers. In response, Alibaba introduced an analytical framework [7] [59] designed to assess execution times and pinpoint bottlenecks specific to various machine learning workloads. The PAI trace from Alibaba encompasses training and inference tasks with their GPU utilization, memory consumption, and other task specifications. The trace indicated that the utilization of P100 GPUs deployed in the data center exhibited a broad spectrum, ranging from 40% to 100%. This illustrates the wide range of utilization levels observed across workloads.

Training scheduling techniques. Various scheduling techniques, such as gang and elastic scheduling, are used in data centers [9]. Gang scheduling requires all GPUs to be allocated simultaneously for a task [60], while elastic scheduling allows flexible GPU usage. Some schedulers employ bin packing or first-fit approaches, considering average memory usage and task utilization [61]. Gandiva [62] ensures resource fairness, while Gavel [63] optimizes for heterogeneity-aware scheduling. MLCloudPrice [64] focuses on cost reduction using linear programming and spot instances for training jobs.

To our best knowledge, `CorrGPU` is the first study that proposes a correlation analysis of GPU utilization to significantly reduce CapEx and ensure that task durations are within the due times.

2.3 Problem Formulation

In this section, we first introduce the notation and correlation analysis, and then present the constrained optimization problem. Finally, we outline the iterative approach used to determine the minimal number of GPUs required. We begin by providing a list of the notation used throughout this Chapter.

- G: Total number of GPUs under consideration.
- L: Total number of tasks.
- m_j: Memory requirement of task j.
- u_j: Normalized utilization of task j.
- a_j: Arrival time of task j.
- d_j: Dispatch time of task j.
- c: Memory capacity of each GPU.
- $\rho_{j,k}$: Correlation coefficient between tasks j and k.
- v: Number of GPUs a task is assigned to (1 for single-GPU training, 2 for multi-GPU training).
- $x_{i,j}$: Binary decision variable, where $x_{i,j} = 1$ if task i is assigned to GPU j, and 0 otherwise.
- y_j: Binary decision variable, where $y_j = 1$ if GPU j is used, and 0 otherwise.
- α: Correlation threshold. If $\rho_{i,k} > \alpha$, tasks i and k cannot be co-located on the same GPU.
- P: Price per GPU.

Correlation Analysis. Before discussing the optimization problem, we define the correlation coefficient between two distributions x and y, denoted $\rho(x, y)$. The

correlation coefficient for two distributions, $x = \{x_1, \ldots, x_n\}$ and $y = \{y_1, \ldots, y_n\}$, is given by:

$$\rho_{xy} = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{\sqrt{n \sum x_i^2 - (\sum x_i)^2} \sqrt{n \sum y_i^2 - (\sum y_i)^2}}. \tag{2.1}$$

The correlation coefficient ρ_{xy} usually ranges from -1 to 1.

Optimization Problem. The optimization problem focuses on minimizing CapEx. The formula for computing CapEx is:

$$\text{CapEx} = \left(\sum_{j=1}^{G} y_j \right) \times P. \tag{2.2}$$

Here, minimizing the number of GPUs effectively reduces CapEx since fewer active GPUs ($y_j = 1$) means lower cost. Instead of directly assuming a very large G, we use an iterative approach to estimate the minimum number of needed GPUs as shown below:

1. Initialize $G = 1$.
2. For a given G, the solver attempts to find a feasible solution to the optimization problem defined in Equation (2.3) with the constraints in Equations (2.4–2.7).
3. If no feasible solution is found, increment G and repeat.
4. Once a feasible solution is found for some $G = G^*$, this G^* is the minimum number of GPUs required.

Objective Function per iteration. For a given G, the optimization objective is to minimize the total number of active GPUs following the approaches used in previous work [65], ensuring that all tasks are scheduled to the available GPUs (G), subject to the constraints listed below.

$$\min \sum_{j=1}^{G} y_j, \tag{2.3}$$

1. **Memory Capacity:** The total memory usage of tasks on each GPU must not exceed its capacity:

$$\sum_{i=1}^{L} m_i \cdot x_{i,j} \leq c \cdot y_j, \quad \forall j \in \{1, 2, \ldots, G\}. \tag{2.4}$$

2. **Correlation:** Tasks that are highly correlated (i.e., $\rho_{i,k} > \alpha$) cannot be placed on the same GPU to avoid simultaneous peaks:

$$x_{i,j} + x_{k,j} \leq 1, \quad \forall (i,k) : \rho_{i,k} > \alpha, \quad \forall j \in \{1, 2, \ldots, G\}. \tag{2.5}$$

   Here α is a user-defined correlation threshold, typically in the range (-1,1).

3. **Task Assignment:** Each task must be assigned to exactly v GPUs:

$$\sum_{j=1}^{G} x_{i,j} = v, \quad \forall i \in \{1, 2, \ldots, L\}. \tag{2.6}$$

   Here $v = 1$ for Single GPU Training and $v > 1$ for Distributed GPU Training.

4. **Dispatching:** All tasks should be dispatched as soon as they arrive.

$$d_i \geq a_i + \epsilon, \quad \forall i \in \{1, 2, \ldots, L\}. \tag{2.7}$$

   Here ϵ is a small positive constant representing the minimal dispatching overhead.

If the solver is unable to find a feasible solution for this optimization problem with the current G, we increase G and repeat the process. The smallest G for which feasibility is achieved, denoted G^*, is the optimal solution.

Computational Complexity and Solver Limitations: We use PuLP [66] to obtain the optimal solution. The solver's tolerance is set to 1×10^{-6}. As the number of tasks (L) increases, the problem complexity grows exponentially. This leads to significantly

Table 2.1: Computation Time for Varying Numbers of Tasks

Tasks	2	4	8	16	32	64	128
Solve Time (s)	0.04	0.07	0.16	0.22	4.92	8.95	80.9

Table 2.2: Number of Inequalities for Different Numbers of Tasks

Tasks	2	4	8	16	32	64	128
Number of Inequalities	0	4	48	224	960	3968	16128

longer solving times and a large number of inequalities, as shown in Table 2.1 and Table 2.2.

The iterative approach and constraints ensure that the smallest feasible number of GPUs, G^*, is identified, serving as an optimal solution for comparison against `CorrGPU`.

2.4 Design of `CorrGPU`

In this section, we first perform a correlation analysis of machine-learning workloads. Next, we present an example illustrating how `CorrGPU` operates. We then develop the `CorrGPU` framework, analyze its complexity and overhead, and conclude by discussing practical considerations.

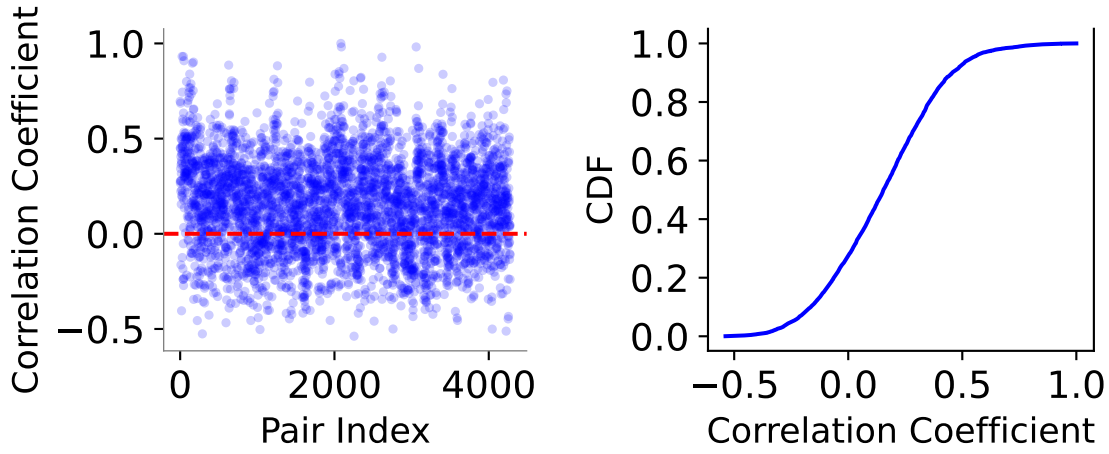
2.4.1 Correlation Analysis of Machine Learning Workloads

GPU utilization exhibits different granularities. GPU utilization from the *nvidia-smi* interface measures the percentage of time over the past sample period during

which one or more kernels are executing on the GPU [67]. At the streaming multiprocessor (SM) [68] level, utilization measures the distribution and execution efficiency of thread blocks across SMs. At the instruction level, utilization is measured as the ratio of successfully issued warp instructions to stalled cycles, indicating the effectiveness of the warp scheduler. Finally, tensor core utilization focuses on the performance of specialized cores for matrix operations, evaluating the dominance and throughput of tensor core instructions.

Although there are multiple methods to compute GPU utilization, coarser-granularity metrics, such as GPU utilization reported by the *nvidia-smi* interface, are preferred in data center scheduling because the overhead of obtaining this information is negligible. Finer-grained metrics, such as SM and tensor core utilization, are helpful for application tuning; however, they add significant overhead and complexity. This Chapter adopts GPU-level utilization, as we focus on cluster-level scheduling; however, it is important to note that `CorrGPU` can also utilize other metrics in its correlation analysis. In this work, we assume the availability of predicted task utilization patterns for all tasks through short-term profiling or predictive modeling.

We investigate both vision and transformer workloads to gain insight into the correlation structure of real-world Machine Learning tasks. The selected workloads include DenseNet [69], Inception [70], MobileNet [71], ResNet [5], GoogleNet [72], VGG [73], Swin Transformer [74], and GPT-2. These workloads are executed on our hardware testbed, where GPU utilization is measured using the '*nvml*' interface [67]. The measured GPU utilization indicates the percentage of time that GPU kernels were active during the previous sampling period. Each workload is executed by fixing the epoch count at 80 and varying the batch size (B) across $10, 20, 30, 40, 50$, and 60.



(a) Correlation Coefficient

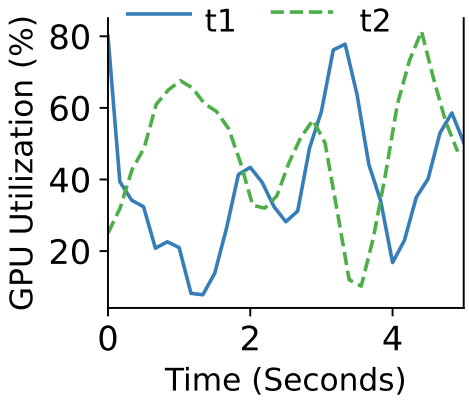
(b) CDF

Figure 2.1: (a) Scatter plot showing the distribution of pairwise correlation coefficients. (b) Cumulative Distribution Function (CDF) plot depicting the overall distribution pattern of the pairwise correlation coefficients.

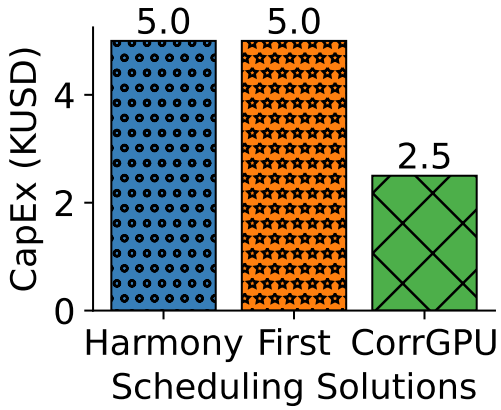
We compute pairwise correlation coefficients using Equation (2.1) on GPU utilization data from the workloads described above. Using correlation analysis is essential because tasks that exhibit a negative correlation do not simultaneously peak in resource utilization. Consequently, these tasks do not compete for GPU resources, enabling more efficient utilization of GPUs for concurrent tasks. The scatter plot in Figure 2.1(a) illustrates the variation in the correlation coefficients for all pairs of workloads, while the cumulative distribution function (CDF) in Figure 2.1(b) shows the distribution of these coefficients. The scatter plot shows that most workload pairs exhibit weak or negative correlations, indicating that their GPU utilization does not peak simultaneously. The CDF in Figure 2.1(b) shows that 75% of the workload pairs have correlation coefficients below 0.25, suggesting a limited overlap in utilization peaks.

2.4.2 Example

We illustrate a simple example to show how `CorrGPU` works. The primary goal of this example is to demonstrate how each baseline functions and to highlight the differences between the proposed solution and the baselines.



(a) GPU Utilization for the motivation example.



(b) CapEx for different scheduling strategies.

Figure 2.2: (a) GPU utilization and (b) CapEx comparison in the motivational example. `CorrGPU` yields the lowest CapEx among all baselines.

The baselines used in this example are `Harmony` and `First`. Both baselines use bin packing to schedule tasks, relying on the utilization threshold (q) as a scheduling metric. The details of the baselines are listed below.

1) Harmony calculates the sum of the maximum utilization of the existing and incoming tasks on the GPU. If the combined peak utilization of the two exceeds the utilization threshold (q), Harmony schedules them to the same GPU [13]. The utilization prediction follows the approach suggested by Yeung et al. [75].

2) First computes the sum of the GPU utilization during the first second for the incoming task and adds it to the aggregated GPU utilization of the tasks already running on the GPU. If their total utilization is less than the threshold (q), the tasks are scheduled to the same GPU.

Performance Metrics. We use two metrics to compare the performance of the proposed solution with the baselines. The first metric is ($CapEx$), defined in Equation (2.2) in Section 2.3. The second metric is Cumulative Task Duration (CTD), defined as the sum of the Job completion times of all tasks. The CTD is represented in Equation (2.8), where L is the set of all tasks, and d_j is the completion time of task j.

$$\text{CTD} = \sum_{j \in L} d_j \tag{2.8}$$

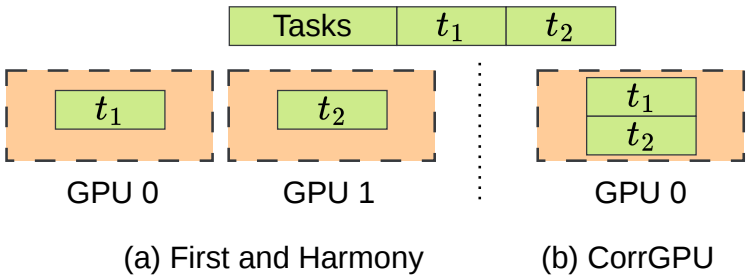


Figure 2.3: `CorrGPU` uses only one GPU and decreases the $CapEx$ for the motivational example.

This example considers two training tasks (t_1) ResNet with a batch size of 40 and (t_2) SqueezeNet with a batch size of 30 as shown in Figure 2.2(a). The average utilizations of t_1 and t_2 are 41.36% and 41.92%, respectively, and their peak utilizations are 80.80% and 61.80%, respectively. The first sample utilization of t_1 and t_2

is 80% and 25%, respectively. During DNN training, because the model architecture remains the same and only the input data changes, GPU utilization can be periodic as we traverse the same layers [37]. Hence, we use a 5-second interval to capture the essential utilization pattern, since it is periodic. We set the utilization threshold (q) at 100% and the correlation threshold (α) at 0. The two tasks are released simultaneously at time $t = 0$ seconds. The scheduling results for `Harmony`, `First`, and `CorrGPU` are listed below. Note that, for simplicity, we do not consider constraints, such as memory requirements, in this example, but the proposed algorithm presented in the next subsection accounts for these constraints.

1. For **Harmony**, the scheduler adds the maximum utilization of t_1 and t_2 (80.80%+ 60.80% = 141.16%), which exceeds the GPU utilization threshold of 100%. As a result, the scheduler assigns t_1 to GPU 0 and t_2 to GPU 1, as shown in Figure 2.3. Harmony thus uses two GPUs.

2. **First** schedules tasks based on the initial utilization in the first second. For t_1 and t_2, the initial utilizations are 80% and 25%, respectively. Their sum (80% + 25% = 105%) exceeds the threshold ($q = 100\%$). Thus, t_1 and t_2 are assigned to GPU 0 and GPU 1, respectively, as shown in Figure 2.3.

3. **CorrGPU** schedules tasks with negative correlation coefficients on the same GPU to minimize resource contention. As soon as t_1 and t_2 arrive, we compute their correlation coefficient (ρ) using Equation (2.1). The correlation coefficient is -0.8, which is below the correlation threshold ($\alpha = 0$). The sum of their average utilization is $(41.36\% + 41.92\%) = 83.28\%$, which is less than the utilization

threshold ($q = 100\%$). Since both criteria are satisfied, `CorrGPU` schedules t_1 and t_2 to GPU 0, as shown in Figure 2.3.

Evaluation on Hardware Testbed. We evaluate the motivational example on the hardware testbed by setting the Nvidia V100 GPU frequency to 735 MHz and running both workloads for 60 *epochs*. For First and Harmony, the scheduling solution is the same since both schedule tasks t_1 to GPU 1 and t_2 to GPU 2 as shown in Figure 2.3. The measured task durations of the tasks for t_1 and t_2 on the testbed are 59.1 and 63.1 seconds for First and Harmony.

We then test our proposed solution, `CorrGPU`, which schedules both tasks (t_1 and t_2) to GPU 1. Here, the measured task durations for t_1 and t_2 are 64.1 and 67.4 seconds, respectively. The task duration for `CorrGPU` is slightly longer than that for First and Harmony. Since the two tasks are negatively correlated, the increase is not greater because they do not compete for resources.

We compare the (*CapEx*) of `CorrGPU`, Harmony, and First using Equation (2.2). We assume $P = 2.5$ K USD as the cost of a single 32 GB V-100 GPU. Figure 2.2(b) shows the *CapEx* for all solutions. Harmony and First require two GPUs, leading to a *CapEx* of $2 \times 2.5 = 5$ K USD. `CorrGPU` requires only one GPU, leading to a *CapEx* of $1 \times 2.5 = 2.5$ K USD. Therefore, `CorrGPU` reduces *CapEx* by 50% compared to Harmony and First.

We can also reduce CapEx by scheduling tasks sequentially on a single GPU. However, this approach can violate the Service Level Agreements (SLAs) with users [76]. Gao et al. [76] survey 103 researchers to understand their tolerance for deadline delays in deep learning training (DLT) jobs with SLA guarantees. The results show that

approximately 3%, 12%, 21%, and 22% of participants accept deadline extensions of 0%, 5%, 10%, and 20%, respectively.

Thus, we use 20% as the threshold for SLO violations (*due time*). We compute the CTD for all solutions using Equation (2.8) and find that the CTD for `CorrGPU` is $64.1 + 67.4 = 131.5$ seconds. The increase in CTD for `CorrGPU` is $131.5 - 122.2 = 9.3$ seconds, which is approximately 7.6% higher than the CTD of the two baselines, which are the fastest among the three solutions. This 7.6% increase remains below the 20% SLA threshold, ensuring that `CorrGPU` complies with SLA requirements while meeting user expectations.

2.4.3 Framework Design

Algorithm 1 details the design steps of `CorrGPU`. The algorithm activates whenever a new task arrives. As mentioned in Section 2.4.2, `CorrGPU` performs correlation analysis and dispatching. The steps performed by `CorrGPU` are listed below.

1. When a new task $t_i \in L$ (where $i > 1$) is released from the task queue, the `CorrGPU` module accesses all the GPUs listed in the *gputracker* (which contains GPUs that have tasks running on them) and evaluate if task (t_i) fits within the available DRAM capacity of GPU g_i (ie, $m_i < C_{\text{avail}}$) as shown in Lines $(3-4)$.

2. If the DRAM capacity condition holds, `CorrGPU` calculates the correlation coefficient δ between the utilization of task t_i and previously scheduled tasks on GPU g_i, as shown on Line 7. Since the correlation coefficient in Equation (2.1) requires vectors of equal length, we compute the minimum of the two task durations and truncate both traces to this common length before computing the coefficient. For upcoming tasks whose full utilization traces are not available,

Algorithm 1 `CorrGPU`

Upon arrival of task t_i

Input: Memory requirement of task m_i, utilization of task u_i, correlation threshold α, distribution factor v

Data: *gputracker*

$buffer \leftarrow []$
$task_scheduled \leftarrow$ False
for $g_j \in gputracker$ **do**
    $C_{avail} \leftarrow$ available DRAM capacity of g_j
    $g_j(u) \leftarrow$ current utilization of GPU (g_j)
    if $m_i \leq C_{avail}$ **then**
        $\delta \leftarrow \rho(g_j(u),\ t_i(u))$
        if $\delta < \alpha$ **then**
            $\delta_1 \leftarrow \mu(g_j(u),\ t_i(u))$
            $buffer \leftarrow \text{push}((\delta,\ \delta_1,\ g_j))$
            $task_scheduled \leftarrow$ True
        end if
    end if
end for
if not $task_scheduled$ **or** $buffer = \emptyset$ **then**
    Allocate a new GPU g_{new}
    $gputracker \leftarrow gputracker \cup \{g_{\text{new}}\}$
    for $k = 1$ **to** v **do**
        $\text{dispatch}(t_i) \rightarrow g_{\text{new}} + k$
    end for
else
    Sort $buffer$ by increasing δ, then increasing δ_1
    $g_{\text{best}} \leftarrow buffer[0].g_j$
    for $k = 1$ **to** v **do**
        $\text{dispatch}(t_i) \rightarrow g_{\text{best}} + k$
    end for
end if

`CorrGPU` relies on *offline profiling*. Specifically, we pre-profile representative utilization traces for each combination of model architecture and batch size. When a new task t_i arrives, its expected utilization trace is retrieved from this offline database and compared with historical tasks on g_i. The correlation is evaluated

over the overlapping window. For instance, if a previously executed task ran for 40 seconds and the profiled trace of t_i spans 30 seconds, the correlation is computed over the first 30 seconds of both traces. `CorrGPU` assumes that job metadata (e.g., architecture, batch size, and training configuration) is available in advance; this is standard in data center ML scheduling pipelines. When the task duration is known, `CorrGPU` uses it directly to define overlap windows. If not, it defaults to average durations based on previously observed tasks with similar characteristics.

3. In Line 8, `CorrGPU` evaluates whether the correlation coefficient δ between task t_i and the tasks running on GPU g_i is below the specified threshold α. In Line 9, `CorrGPU` calculates δ_1, the average utilization between the new task t_i and the aggregate utilization of all tasks already running on GPU g_i. Importantly, this utilization value is treated as a soft constraint rather than a strict hardware limit. Since GPU utilization is dynamic, occasional overlaps that slightly exceed GPU capacity may introduce minor execution delays but typically do not result in failure or instability.

4. In line 8, once a GPU g_j satisfies the memory availability and correlation threshold criteria, the algorithm computes the mean utilization δ_1 between the new task t_i and the set of tasks currently running on g_j. It then stores a tuple $(\delta,\ \delta_1,\ g_j)$ in the *buffer*. This buffer serves as a temporary holding list of candidate GPUs eligible for task placement.

5. If the *buffer* is empty, it indicates that no existing GPUs that have tasks running on them can satisfy the scheduling constraints. This situation might arise

from insufficient DRAM capacity or if the correlation coefficient exceeds the threshold. In this case, as illustrated in lines 15–19, a new GPU is dynamically allocated to t_i. The new GPU is added to the *gputracker*, and t_i is dispatched there. If $v = 1$, the task goes to a single GPU; if $v > 1$, it is distributed among multiple GPUs, beginning with g_{new} and increasing the indices as $g_{\text{new}+k}$.

6. If the *buffer* contains items, the scheduler sorts the *buffer* to choose the GPU with the lowest correlation coefficient (line 22) and mean utilization. The first GPU in the sorted list is selected for dispatch. Using the distribution factor (v), t_i is spread across GPUs starting from the selected GPU g_j, as shown in lines 25.

2.4.4 Complexity Analysis

The time complexity of *Algorithm 1* is determined by three main operations: iterating over tasks, evaluating candidate GPUs, and sorting the candidate buffer. For n tasks, `CorrGPU` dynamically allocates GPUs as needed, with the number of active GPUs at any point denoted by g, where $g \leq n$. The outer loop iterates over all n tasks, contributing $O(n)$. For each task, the algorithm examines up to g GPUs stored in *gputracker*, resulting in an additional $O(g)$ per task. This leads to a task-GPU iteration cost of $O(n \times g)$. Additionally, for each task, `CorrGPU` sorts a buffer of up to g candidate GPUs based on correlation scores, contributing $O(g \log g)$ per task. Across all tasks, this results in a total cost of $O(n \times g \log g)$. Combining both contributions, the overall time complexity is $O(n \times g + n \times g \log g) = O(n \times g \log g)$. In the worst-case scenario where each task is assigned to a unique GPU (i.e., $g = n$), the complexity becomes $O(n^2 \log n)$

2.4.5 Practical Considerations

`CorrGPU` assumes the availability of predicted task utilization patterns through short-term profiling or predictive modeling. `CorrGPU` tolerates small inaccuracies as long as the broad temporal patterns are preserved. `CorrGPU` tolerates small inaccuracies as long as the relative timing and magnitude of utilization fluctuations are preserved. Our evaluations (Sections 2.6 and 2.7) confirm that `CorrGPU` remains effective even under real-world traces.

`CorrGPU` may be less beneficial when workloads are highly synchronized or when accurate utilization traces are unavailable. In such cases, prediction errors in GPU utilization may lead to a slight degradation in scheduling performance. While `CorrGPU` is designed to exploit weak or negative correlations under typical ML training traces, real-world data centers often encounter workload surges, shifting deadlines, and unpredictable arrival patterns. Evaluating `CorrGPU` under such bursty scenarios is left for future work.

Currently, `CorrGPU` performs correlation analysis only on GPU utilization and treats other resource demands, such as memory capacity, as hard constraints. It is essential to note that `CorrGPU` can be extended to perform correlation analysis on memory bandwidth and interconnect traffic, thereby transforming the current one-dimensional scheduling model into a multidimensional scheduling model. However, such an extension may significantly increase computational complexity and yield only a small improvement, as the memory requirements of ML workloads are generally more stable than their GPU utilization.

2.5 Experimental Setup

Hardware Testbed. Our testbed consists of six Nvidia V100 GPUs. We use Ubuntu 20.04 LTS as the server OS and the CUDA Toolkit 11.6. We use PyTorch 1.12.1. as the ML framework. For hardware evaluation, we conduct scheduling online and dispatch offline. To enable spatial sharing, the CUDA Multiprocessing Service (MPS) is used. The MPS thread percentage is fixed at 100% to ensure full GPU provisioning. All workloads from Section 2.4 are incorporated into this evaluation. We adjust the batch sizes of the workloads to align with those specified in the PAI [7] trace. The workloads for the experiments are as follows: t_1 corresponds to DenseNet [77] with a batch size of 40. Similarly, t_2 is Vision Transformer (ViT) with a batch size of 40, t_3 is SqueezeNet [69] with a batch size of 40, and t_4 is ResNet [5] with a batch size of 40. The subsequent workloads include t_5, which is MobileNet [71] with a batch size of 40, t_6 as MnasNet [78] with a batch size of 40, t_7 as GoogleNet [72] with a batch size of 30, and t_8, which is VGG16 [73] with a batch size of 40. All these workloads are trained on a Kaggle dataset of wild animal images.

Trace-driven Simulation. To address the limitations of our hardware testbed, we developed a Python simulator to evaluate `CorrGPU` on a large scale using real-world traces. The simulator is validated by ensuring that it reproduces the results of our testbed in smaller-scale experiments, thereby confirming its accuracy. The simulator is built from the open-source code of Gavel [63] and is designed to emulate a heterogeneous data center. The simulator features several key components: a simulated clock module, a correlation analysis module, a job dispatcher, and a GPU manager. Its primary framework operates in a for loop, where each iteration simulates one second of the trace. Before execution, the jobs in the trace are sorted by

start time in ascending order. At each iteration, the job dispatcher checks each job's start time and dispatches it if the current simulated time matches or exceeds the job's start time. Jobs are assigned to GPUs using the first-fit bin-packing algorithm, which considers GPU utilization and memory size constraints. The duration of each task is determined using a linear model based on curve fitting. In addition, the simulation computes metrics such as the average number of GPUs used for training and estimates CapEx. Once a job is completed, it is removed from the host GPU's job list.

2.6 Hardware Evaluation

In this section, we evaluate the performance of `CorrGPU` against the baselines described in Section 2.5. Importantly, this experiment is designed to demonstrate `CorrGPU`'s robustness to model error, showing that our correlation-aware scheduler remains effective even when task utilization predictions are noisy or inaccurate. We generate synthetic GPU utilization traces by sampling, for each task, from a Gaussian distribution whose mean (μ) and variance (σ^2) match those of real ML training tasks in the PAI trace [7].

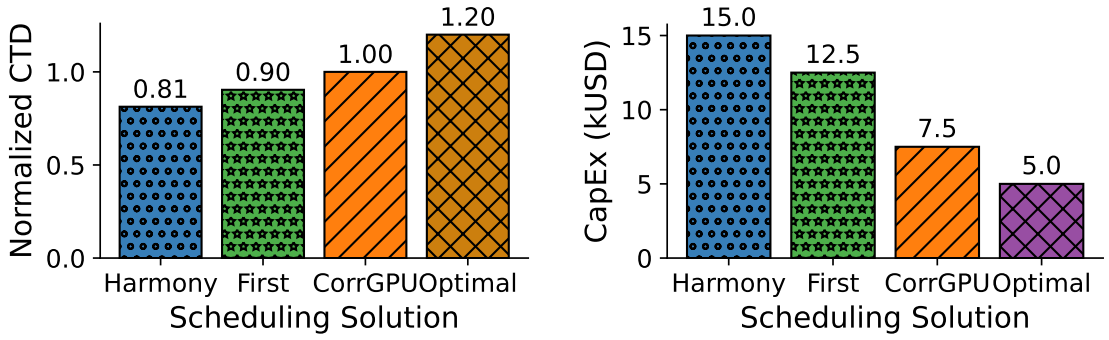
We select 14 ML tasks from the PAI trace based on their GPU utilization and memory requirements. Tasks arrive dynamically in two bursts. In stage 1, tasks t_1, t_2, t_3, t_4 arrive at 0, 1, 2, and 3 seconds, and tasks t_5, t_6, t_7, t_8 arrive at 20, 21, 22, and 23 seconds. In stage 2, tasks $t_9, t_{10}, t_{11}, t_{12}$ arrive at 400 seconds and tasks t_{13}, t_{14} at 420 and 422 seconds. Tasks t_9–t_{14} replicate the workloads of t_1–t_8 respectively (e.g. t_9 uses t_1's workload, t_{10} uses t_2's, etc.). We repeat each configuration five times and report the mean results with one standard deviation.

In the first step, we measure the number of GPUs used throughout the experiment. `CorrGPU` requires 3 GPUs, compared to 5 for *First*, 6 for *Harmony*, and 2 for *Optimal*. Next, we compare the *CapEx* and normalized *CTD* for all baselines and `CorrGPU`. We normalize to `CorrGPU`. The normalized *CTD* for all solutions appears in Figure 2.4(a). The normalized *CTD* of *Harmony* is 0.81, and for *First*, it is 0.90. Gao et al. [76] survey 103 researchers to understand their tolerance for deadline delays in deep learning training (DLT) tasks with SLA guarantees. The results show that approximately 3%, 12%, 21%, and 22% of participants accept deadline extensions of 0%, 5%, 10%, and 25%, respectively. Thus, we use 25% as the threshold for SLO violations (*due time*). `CorrGPU`'s *CTD* is 1.0, which stays within 1.25× *Harmony*'s *CTD*. This shows that CorrGPU meets its *due time*. *Optimal*'s *CTD* is slightly higher because it uses fewer GPUs than `CorrGPU`.

`CorrGPU` has the second smallest *CapEx* as shown in Figure 2.4(b) since it uses fewer GPUs. Compared to *Harmony*, `CorrGPU` saves 50% of *CapEx*. Compared to *First*, `CorrGPU` saves 40% of *CapEx*. *Optimal*'s *CapEx* is smaller since it uses fewer GPUs and the solver finds the most optimal solution. This is not feasible when the number of tasks increases, as stated in Section 2.3.

2.7 Simulation Evaluation

As introduced earlier, `CorrGPU` estimates the minimum number of GPUs required and does not significantly increase the CTD of the tasks, as shown in Section 2.6. We perform large-scale simulations to determine the optimal number of GPUs required for `CorrGPU`. We compare `CorrGPU` against the baselines, First and Harmony. As mentioned earlier, we do not compare our results with the optimal solution for the



(a) CTD

(b) CapEx

Figure 2.4: Hardware evaluation using the PAI trace. (a) `CorrGPU`'s *CTD* is smaller than that of *Optimal* and remains within the due time. (b) `CorrGPU` uses fewer GPUs compared to *Harmony* and *First*, resulting in a lower *CapEx*.

simulation due to the complexity described in Section 2.3. In this section, we perform a trace evaluation, assessing our solution using the PAI trace comprising of 5000 tasks.

In this experiment, we test the PAI trace after the preprocessing mentioned in Section 2.5. Figure 2.5 illustrates the variation in the average number of GPUs used for the experiment. `CorrGPU` utilizes fewer GPUs compared to the baselines, namely First and Harmony. On average, Harmony uses 23.2 GPUs, First uses 16.7 GPUs, while `CorrGPU` requires only 15.32 GPUs. This corresponds to a reduction in GPU usage of approximately 34% compared to Harmony and 8.3% compared to First.

In Figure 2.5, we observe a significant increase in GPU usage across all baselines during the first 200 tasks. This spike is attributed to the dispatch of a large volume of tasks that have GPU utilization values exceeding 80%, necessitating the assignment of separate GPUs for each task. Furthermore, during this period, the correlation values remain positive, further contributing to the increase in GPU usage in all baselines,

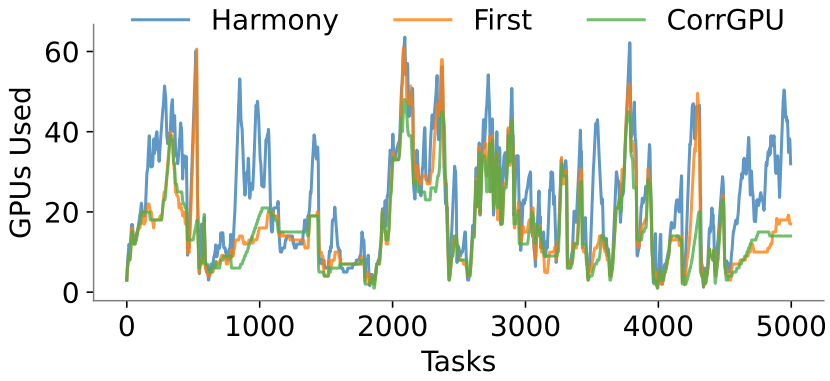
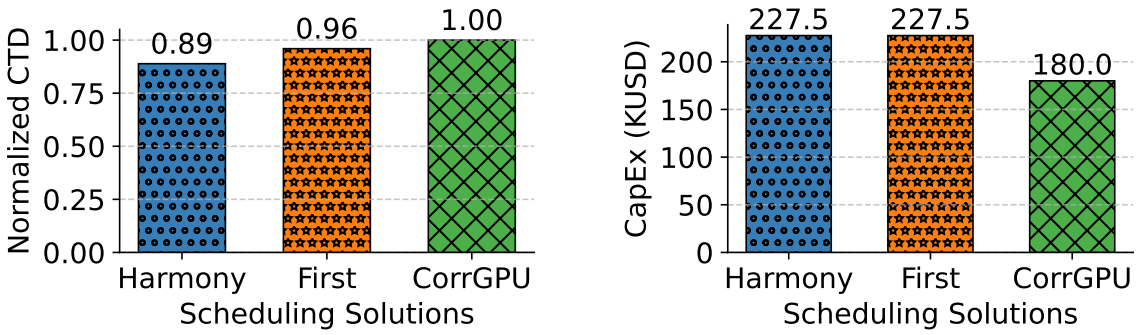


Figure 2.5: `CorrGPU` uses fewer GPUs compared to the baselines for the large-scale simulation using the PAI trace.

including `CorrGPU` . Reducing the number of GPUs must not increase task durations beyond the *due times*.

As shown in Figure 2.6(a), `CorrGPU` demonstrates a slightly higher *CTD* compared to the baseline approaches, *Harmony* and *First*, which have *CTD* values of 0.89 and 0.96, respectively. In contrast, *CorrGPU's CTD* is 1.02, which remains within the due time threshold of $1.25 \times 0.89 = 1.11$, where 0.89 is *Harmony's* CTD. `CorrGPU` reduces resource contention by consolidating tasks with negative correlations and minimizing contention.

Next, we evaluate the (*CapEx*) for each scheduling solution using Equation (2.2). Following the methodology outlined in Algorithm 1, we count every GPU assigned at least one task throughout the trace, reflecting realistic provisioning requirements. As shown in Figure 2.6(b), both Harmony and First require 91 unique GPUs, corresponding to a *CapEx* of \$227.5K. In contrast, `CorrGPU` uses only 72 GPUs, resulting in a significantly lower *CapEx* of \$180.0K. This represents a cost reduction of approximately 20.88% compared to both baselines. The savings arise from *CorrGPU's*



(a) CTD

(b) CapEx

Figure 2.6: (a) CorrGPU exhibits a slightly higher Cumulative Task Duration (CTD) than Harmony (0.89) and First (0.96), but remains within the acceptable due times. (b) CorrGPU achieves significantly lower *CapEx* than both baselines by reducing the total number of unique GPUs provisioned.

ability to leverage task correlation and utilization patterns to co-locate tasks more effectively and minimize redundant GPU usage.

2.8 Conclusion

The rapidly growing demand for MLaaS services has been driving the construction of new GPU data centers. However, an important question for many data center operators is how to minimize their investment in CapEx, while managing to schedule all the expected ML workloads without causing undesired long delays. This Chapter presents CorrGPU, a correlation-aware GPU scheduling algorithm that considers the utilization correlation among different tasks to minimize the number of needed GPUs in a data center. CorrGPU is designed based on a key observation from the analysis of real ML traces that different tasks do not have their GPU utilization peak at exactly the same time. Our simulation results on real-world ml traces also show that `CorrGPU` outperforms several state-of-the-art solutions by reducing capital expense

by 20.88%. As part of our future work, we plan to conduct a sensitivity study on the correlation threshold parameter to evaluate the trade-off between GPU savings and task latency. Furthermore, we intend to extend the evaluation of `CorrGPU` to large-scale LLM-based training tasks.

Chapter 3: CATS

3.1 Introduction

As discussed in Chapter 1, beyond GPU provisioning, a critical challenge in modern GPU data centers is reducing runtime power consumption while maintaining acceptable performance for machine learning (ML) workloads. Although GPUs are highly efficient for parallel computation, large-scale GPU clusters consume substantial power, leading to significant operational expense (OpEx). Therefore, improving runtime efficiency is essential for sustainable and cost-effective data center operation.

Existing GPU power optimization approaches primarily rely on techniques such as dynamic voltage and frequency scaling (DVFS) or offline profiling. However, DVFS alone cannot eliminate idle power consumption, which remains significant when GPUs are underutilized. Moreover, prior scheduling approaches often fail to capture the dynamic behavior of ML workloads, as they typically assume constant GPU utilization. As highlighted in Chapter 1, real-world ML workloads exhibit time-varying utilization patterns, creating opportunities for more efficient scheduling.

In this work, we propose `CATS`, a power optimization framework specifically designed for GPU data centers that run ML training tasks. `CATS` leverages *correlation-aware scheduling*, which analyzes GPU utilization patterns among various ML tasks

to efficiently consolidate them onto fewer GPUs. By scheduling tasks with complementary resource demands, we can fully utilize active GPUs and put unused GPUs to sleep, thereby saving idle power. `CATS` is also integrated with dynamic GPU frequency scaling to maximize dynamic power savings without much impact on the JCTs of ML training tasks. `CATS` focuses on scheduling training tasks, since inference tasks typically have very short durations, usually in milliseconds [54] [52] [55]. Therefore, we focus on scheduling **training tasks** with longer durations [23] [22] and provide multiple data points for correlation analysis. To this end, we first model the impact of frequency scaling on GPU power consumption and task JCTs. We then formulate GPU consolidation as an NP-hard constrained optimization problem. Then, we derive an *optimal* solution that determines the best consolidation strategy. To significantly reduce computational overhead, we design a heuristic algorithm to efficiently consolidate **ML training tasks** based on the correlations observed in their GPU utilization. Finally, we discuss practical considerations for implementing `CATS` in real-world data centers, including scalability, GPU utilization prediction, and integration with existing infrastructure.

The major contributions of this work are as follows:

- We examine real-world traces and observe that ML tasks generally exhibit weak pairwise correlations and do not peak simultaneously. Leveraging this insight, we propose correlation-aware scheduling that strategically consolidates ML training tasks to minimize the number of required GPUs, thereby reducing overall power consumption.

- We formulate ML task consolidation as a constrained optimization problem. Given the NP-hard nature of this problem, we mathematically derive an *optimal*

solution. Although this solution is computationally intensive and impractical for real-time data center operations, it serves as an upper bound to guide the development of efficient heuristic algorithms.

- We design `CATS`, a novel GPU scheduling algorithm that exploits the utilization correlations among ML tasks to minimize the power consumption of a GPU data center. `CATS`'s performance is close to that of the optimal solution with significantly reduced computational overhead, making it suitable for practical deployment in real data centers.

- We validate the effectiveness of `CATS` through experiments on our GPU hardware testbed and trace-driven simulations. Our testbed results demonstrate a substantial reduction in the number of required GPUs. Furthermore, simulation evaluations show that `CATS` outperforms several baseline solutions, achieving up to an 24.7% decrease in power consumption.

The remainder of this chapter is structured as follows: Section 3.2 discusses the related work. In Section 3.3, we formalize the optimization problem. Section 3.4 motivates utilization-based correlation and details the design of the `CATS` framework. Section 3.5 describes the experimental setup. Hardware and simulation evaluations are covered in Sections 3.6 and 3.7, respectively. Finally, Section 3.8 summarizes our conclusions.

3.2 Related Work

The related work can be classified into two categories: (1) Schedulers for AI Training in data centers, and (2) Power optimization for GPU clusters.

Schedulers for AI Training in data centers: Early efforts used cluster managers like Kubernetes or YARN to schedule DL jobs in the cloud without considering the characteristics of DL jobs, which results in low performance [8] [79]. Recent efforts proposed specialized cluster schedulers for DL training jobs [10] [9] [54] [61] [50] [80]. These efforts focus on optimizing for JCT, fairness, or GPU utilization while ignoring the energy consumption of DL jobs and the GPU cluster.

Power optimization for GPU clusters. Prior research on power optimization has primarily focused on CPU-based systems, where strategies such as dynamic power management, server consolidation, and CPU dynamic voltage and frequency scaling (DVFS) have been extensively investigated [81–83]. However, with the rapid growth of AI workloads, recent studies have begun to focus on GPUs, which are significant contributors to data center energy consumption due to their high power density and substantial idle power. Consequently, several GPU-focused initiatives have explored DVFS-based power control [14, 56, 84–86], strategies for workload deferral to sustain high GPU utilization [87], and predictive techniques for powering GPU nodes on and off to mitigate idle energy waste [44]. Complementing these node-level optimizations, NuclearDC [88, 89] introduces a framework that couples offline refueling planning with online runtime control to further reduce the operational power cost and grid dependence of GPU-heavy facilities under time-varying demand and grid conditions. More recent work has additionally investigated energy-aware GPU scheduling and dynamic configuration tuning for deep learning training workloads [37, 42, 90]. All the above solutions assume GPU utilization is static, overlooking temporal fluctuations or task correlations. This oversight can lead to suboptimal consolidation decisions and unnecessary GPU or server activation.

3.3 Problem Formulation

In this section, we develop a model to analyze how GPU frequency scaling affects the power consumption and JCT of ML training tasks. We then propose an *optimal* solution to optimize data center power consumption without degrading performance. Finally, we discuss the practical challenges associated with computing this optimal solution.

3.3.1 Modeling

Power Model

We first introduce the power model and then formulate the power optimization as a constrained problem. We use the following notations:

- D: Datacenter.
- N: Total number of servers in D.
- w_i: Total number of GPUs on server i.
- G_{ij}: The jth GPU on the ith server.
- P_{gs}: Per-GPU static power (idle) when a GPU is powered on (W).
- α: Linear coefficient of dynamic power vs. frequency (W/MHz).
- f_{ij}: SM (Streaming Multiprocessor) frequency of GPU j running on server i (MHz).
- P_i^{static}: Server i's static power draw when the server is on (independent of GPU load) (W).

- c_{ij}^{mem}: DRAM capacity of GPU j on server i.
- L: Total number of tasks.
- d_l: Memory consumption of task t_l.
- v_l: Number of GPUs required by task t_l.
- κ_l: Maximum allowed Job Completion Time (JCT) extension factor for task l (e.g., $\kappa_l = 1.2$ for a 20% tolerance).
- $x_{lij} \in \{0, 1\}$: 1 if task l is assigned to GPU j on server i, 0 otherwise.
- $y_{ij} \in \{0, 1\}$: 1 if GPU j on server i is active, 0 otherwise.
- $z_i \in \{0, 1\}$: 1 if server i is active, 0 otherwise.

The power consumption of a GPU consists of both static (idle) and dynamic components. The static power consumption is the baseline power drawn by the GPU when idle, including components such as the power supply and fans. The dynamic power consumption varies with workload and is directly proportional to the GPU's operating frequency. Thus, the total power consumption of a GPU is represented in Equation (3.1), where P_{gpu} is the total power consumption in Watts. P_{gs} represents the static power consumption (obtained from measurements of the GPU's power draw at idle), α is the coefficient for linear dynamic power contribution, and f is the GPU's operational frequency in Megahertz.

$$P_{\text{gpu}} \;=\; P_{gs} \;+\; \alpha\, f. \tag{3.1}$$

For a datacenter (D) comprising a homogeneous GPU cluster consisting of N servers, where each server has w_i GPUs, the total power consumption is shown in

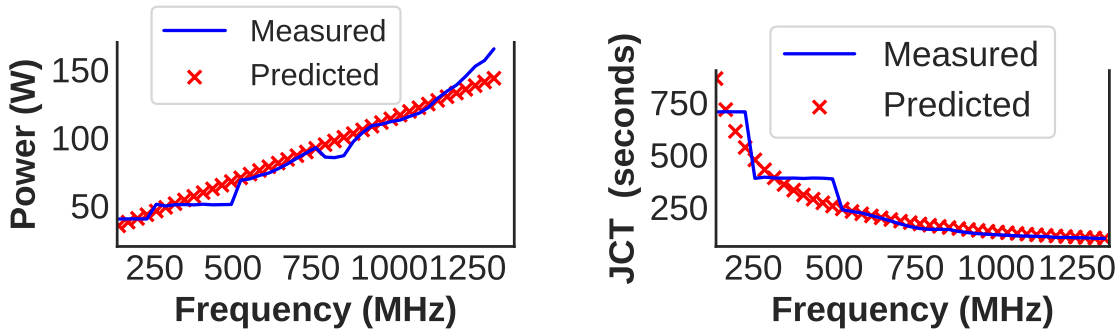
Equation (3.2). To derive the linear coefficient in Equation (3.2), we characterize (P_{gs}, α) for our NVIDIA V100 GPUs by running ML training workloads, specifically *ResNet50* and *VGG16* with a batch size of 60, on frequencies ranging from 135 MHz to 1350 MHz. For each frequency setting, we measure power consumption per training epoch and compute the average across workloads. The GPU power model is expressed in Equation (3.3). In this model, 23.3 W represents the static power consumption, obtained from measurements of the GPU's idle power draw, while 0.09 W/MHz is the coefficient that captures the linear contribution of dynamic power. This model achieves a high degree of accuracy, with an R^2 score of approximately 0.95, as illustrated in Figure 3.1(a).

$$P_{\text{gpu,total}} \;=\; \sum_{i=1}^{N} \sum_{j=1}^{w_i} y_{ij} \left(P_{gs} + \alpha \, f_{ij} \right). \tag{3.2}$$

$$P_{\text{gpu,total}} \;=\; \sum_{i=1}^{N} \sum_{j=1}^{w_i} y_{ij} \left(23.3 \;+\; 0.09 \, f_{ij} \right). \tag{3.3}$$

A typical server used for ML training features multiple GPUs (often 1–8 [54, 91, 92]). The total *server* power for server i includes its static base draw when active, the sum of its GPU power, and the power drawn from other components as shown in Equation (3.4).

$$P_{\text{server},i} \;=\; \begin{cases} P_i^{\text{static}} \;+\; \displaystyle\sum_{j=1}^{w_i} P_{\text{gpu},ij}(f_{ij}) + P_{other} & \text{if } z_i = 1, \\ 0, & \text{if } z_i = 0 \end{cases} \tag{3.4}$$



(a) Power consumption

(b)Job completion time

Figure 3.1: Measured and predicted GPU power consumption and job completion time (JCT) under different SM frequencies: (a) power consumption and (b) JCT.

In Equation (3.4), P_{other} captures the non-GPU power components, including CPU power, memory power, chipset power, and power consumption of the other components. Since ML training tasks predominantly use GPUs, GPUs are the primary contributors to the overall power consumption.

In cloud services, providers often anticipate user requests for machine learning services. To process these requests promptly and satisfy service-level agreements (SLAs), providers keep GPUs active and operate them at their maximum frequency to avoid latency violations [35]. However, keeping GPUs continuously powered on to meet latency requirements results in significant idle power consumption when no requests are being processed. For example, on an A100-PCIE-40GB (250W TDP), the idle power draw for the smallest compute slice is already around $40 - 50$ W. For an H100-PCIE-80GB (350W TDP), the idle power draw for the smallest slice is even higher, at approximately 70-80 W [93]. Moreover, the idle power percentage will increase for future GPUs. Therefore, when GPUs are not needed to serve active tasks, unused GPUs can be transitioned into low-power sleep states to reduce idle

power consumption [44]. In addition, when no tasks are assigned to any GPU on the server (i.e., $z_i = 0$), the server can be powered off or put to sleep to further reduce its static power consumption.

Latency Model

Training tasks do not have strict Service Level Agreements (SLAs) as inference tasks [54]. However, training latency cannot be extended indefinitely. The GPU frequency affects training latency [90]: reducing it can reduce power consumption but may prolong the JCT [10, 76]. Gao et al. [76] conducted a survey involving 103 researchers to assess their tolerance for deadline extensions in Training tasks. The findings revealed that approximately 3%, 12%, 21%, and 22% of respondents were willing to accept deadline extensions of 0%, 5%, 10%, and 20%, respectively. In this chapter, we establish a 20% threshold for determining SLA violations. To understand the impact of frequency scaling on JCT, we analyze how the GPU frequency affects JCT. By varying the frequency of the GPU as described in Section 3.3.1, we measure the average JCT between workloads in Section 3.3.1 and fit a curve to model the relationship between the two as shown in Equation (3.5).

$$e_l(f) \;=\; e_{\min,l} \;\times \Big(\frac{f_{\max}}{f}\Big)^{\beta} \tag{3.5}$$

Through curve fitting, we obtain $\beta = 0.91$, which captures the nonlinear scaling behavior and yields $R^2 \approx 0.97$ (Fig. 3.1(b)).

3.3.2 Problem Formulation

Deep learning training clusters are typically constrained by the *power budget* of the data center rather than the total energy consumed by individual tasks. Consequently, our objective is to minimize the *instantaneous* power draw while ensuring

that each training task does not exceed its SLA. We emphasize that this work optimizes *power* (Watts), because instantaneous power directly affects the rack-level electrical provisioning, cooling capacity, and cluster-wide admission control.

The optimization problem is shown in Equation (3.6).

$$\min \sum_{i=1}^{N} P_{\text{server},i} \tag{3.6}$$

$$\text{s.t.} \quad \sum_{i=1}^{N} \sum_{j=1}^{w_i} x_{lij} = v_l, \qquad \forall l, \tag{3.7}$$

$$\sum_{l=1}^{L} d_l \, x_{lij} \leq c_{ij}^{\text{mem}} \, y_{ij}, \qquad \forall i, j, \tag{3.8}$$

$$x_{lij} \leq y_{ij}, \qquad \forall l, i, j, \tag{3.9}$$

$$z_i \geq y_{ij}, \qquad \forall i, j, \tag{3.10}$$

$$z_i \leq \sum_{j=1}^{w_i} y_{ij}, \qquad \forall i, \tag{3.11}$$

$$x_{lij},\, y_{ij},\, z_i \in \{0, 1\}, \qquad \forall l, i, j \tag{3.12}$$

$$f_{\min} \, y_{ij} \leq f_{ij} \leq f_{\max} \, y_{ij}, \qquad \forall i, j, \tag{3.13}$$

$$f_{ij} \geq f_{\max} \, \kappa_l^{-1/\beta} - M(1 - x_{lij}), \qquad \forall l, i, j \tag{3.14}$$

where $M \geq f_{\max}$.

Consolidating tasks onto fewer GPUs reduces the number of active GPUs (i.e., those with $y_{ij} = 1$), which in turn enables more servers to be powered down/put to sleep (so $z_i = 0$). Constraint (3.7) denotes the distributive constraint used to determine the number of GPUs assigned to a training task. If $v > 1$, distributed training is performed. Equation (3.8) ensures that the task's memory consumption does not exceed the DRAM capacity of the GPU. Constraints (3.9), (3.10), (3.11) represent the activation constraints. Constraint (3.13) specifies the operating frequency range of the GPU. Constraint (3.14) enforces the SLA by ensuring $e_l(f_{ij}) \leq \kappa_l e_{\min,l}$, where

M ($\geq f_{\max}$) is a large constant used to deactivate the constraint when task l is not assigned to GPU G_{ij}. The SLA constraint introduced in Equation (3.14) ensures that frequency scaling does not degrade training performance beyond the specified tolerance.

Observation 1: The server power optimization problem is NP-hard.

Proof. We prove NP-hardness by reduction from the classical *Bin Packing* problem. Given items with sizes $s_\ell \in (0, 1]$ and unit-capacity bins, we create an equivalent instance of our problem. For each item ℓ, define a task with memory demand $d_\ell = s_\ell$ and GPU requirement $v_\ell = 1$. Each server has a single GPU ($w_i = 1$) with unit memory capacity ($c_{ij}^{\text{mem}} = 1$), static power $P_i^{\text{static}} = 1$, and zero dynamic power ($P_{\text{gpu},ij}(f_{ij}) = 0$). In this setting, the memory constraint ensures that the total size of tasks on a GPU does not exceed its unit capacity. The linking constraints activate a server whenever a task is placed on it. Thus, the objective $\min \sum_{i=1}^{N} z_i$ is precisely the problem of minimizing the number of active servers, which corresponds to reducing the number of bins in the *Bin-Packing* problem. Since *Bin-Packing* is NP-hard, our problem is also NP-hard.

3.4 Design of `CATS`

In this section, we begin by discussing GPU utilization and analyzing the correlations in GPU utilization across ML training tasks. We then discuss the consolidation analysis of ML tasks. Building on this analysis, we detail the design and functional components of the `CATS` framework. We then discuss the practical considerations associated with `CATS`.

3.4.1 GPU Utilization

There are different granularities of utilization in GPUs. The GPU utilization measured by the `nvml` interface represents the percentage of time over the past sample period during which one or more kernels are executing on the GPU [67]. At the streaming multiprocessor (SM) [68] level, utilization measures the distribution and execution efficiency of thread blocks across SMs. At the instruction level, utilization is calculated as the ratio of successfully issued warp instructions to stalled cycles, indicating the warp scheduler's effectiveness. Finally, tensor core utilization focuses on the performance of specialized cores for matrix operations, evaluating the dominance and throughput of tensor core instructions. Although finer-grained utilization is useful for application tuning, it incurs significant profiling overhead. In contrast, GPU and SM-level utilization is relatively easy to obtain and is therefore widely used in data center schedulers [68]. To obtain temporal GPU utilization information for our analysis, we collect periodic utilization samples using the NVIDIA Management Library (`nvml`). Specifically, utilization is sampled asynchronously by a dedicated background thread that invokes `nvml` APIs at regular intervals. This approach provides low-overhead access to GPU utilization without interfering with the training execution. While `nvidia-smi` can also be used to query the same `nvml` interfaces, running it as a subprocess adds unnecessary overhead; therefore, our implementation interacts directly with `nvml` in-process. Although more advanced profilers such as NVIDIA Nsight Systems (nsys) can capture extremely fine-grained information (e.g., kernel-level traces, memory transactions, and tensor-core activity), they introduce

substantial runtime overhead and are not suitable for continuous or online monitoring in production training clusters. Our coarse-grained, low-overhead sampling is consistent with how GPU utilization is monitored in real-world cloud environments.

GPU utilization prediction has been extensively studied before [44, 75, 94–97]. For example, some research has achieved a low prediction error, with a Root Mean Squared Logarithmic Error (RMSLE) of 0.154 [75]. Since utilization prediction is *not* the focus of this work, we assume that GPU utilization can be predicted using such existing techniques. It is also important to note that CATS can still work effectively even when the predicted GPU utilization is inaccurate (See Section 3.6). While task memory usage may vary with batch size or degree of parallelism, both parameters are available at the submission time, and the scheduler can account for them when estimating memory demand. Since CATS relies on coarse-grained correlation trends rather than exact point predictions, moderate deviations do not significantly change scheduling decisions. Inaccurate predictions may slightly increase latency or reduce consolidation, while remaining superior to correlation-agnostic baselines.

3.4.2 Utilization Correlation Analysis

To motivate the design methodology of `CATS` in the subsequent sections, we present our analysis of the Philly trace from Microsoft's data center [8]. The trace includes training tasks from production groups that develop products using models for image classification, speech recognition, and other applications. The trace is collected from clusters equipped primarily with Nvidia $P-100$ GPUs. We select 600 tasks from the middle of the trace. We map each task's machine ID in the job trace file to the

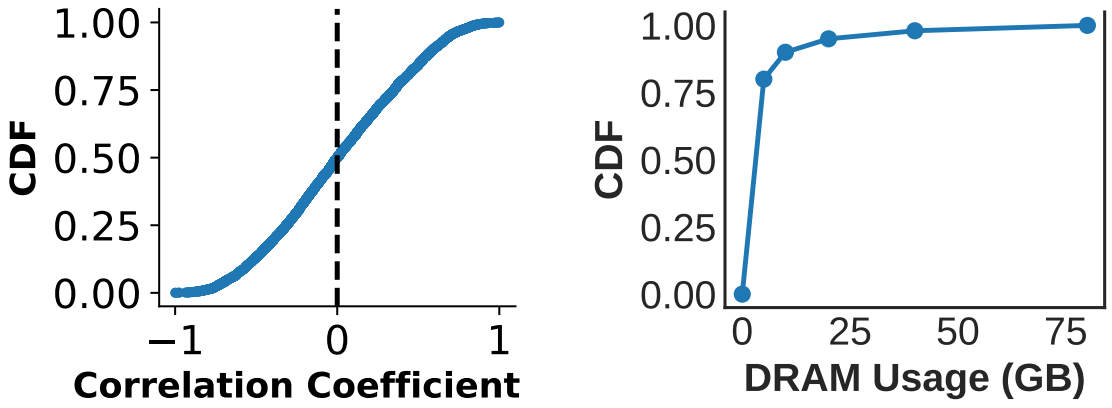
corresponding machine IDs in the GPU utilization trace file. We calculate the aggregate GPU utilization for each job by summing utilization across all GPUs assigned to the task for each minute of execution. For the selected 600 tasks, we compute the correlation coefficient from the utilization values to quantify the similarity in GPU utilization patterns. To quantify the correlations among tasks, we compute the Pearson correlation coefficient for each pair of tasks, as shown in Equation (3.15). The correlation coefficient for two distributions, $x = \{x_1, \ldots, x_n\}$ and $y = \{y_1, \ldots, y_n\}$, is given by:

$$\rho(x, y) = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}, \tag{3.15}$$

where $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$ and $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ are the sample means of x and y, respectively. The correlation coefficient ρ_{xy} usually ranges from -1 to 1. We plot the Cumulative Distribution Function (CDF) of the correlation coefficient for all pairs of tasks in the *Philly trace*, as shown in Figure 3.2(a). From this figure, we observe that 46% of task pairs exhibit negative correlations, indicating that tasks are negatively correlated. Meanwhile, 34% fall between 0 and 0.3, suggesting a weak positive correlation. Overall, approximately 80% of task pairs exhibit correlations $|r_{xy}| \leq 0.3$, indicating that most task pairs are only weakly correlated. Only a small fraction of task pairs show stronger correlations, with $r_{xy} > 0.3$.

Observation 1: The GPU utilization of most training tasks is loosely correlated, so they usually do not peak at the same time.

It is important to note that the correlation values reported here are computed from utilization samples extracted from the trace. Consequently, the metric captures



(a) CDF for GPU Utilization.

(b) CDF for DRAM consumption.

Figure 3.2: (a) The CDF of the correlation coefficient for the 600 tasks chosen from the Philly trace. (b)The CDF of DRAM usage for the 600 selected tasks from the GPUs in the PAI trace.

coarse-grained temporal trends rather than fine-grained variations at the level of kernels, warps, caches, or memory-access behavior. We acknowledge this limitation; however, our objective is not to model microarchitectural contention. Instead, correlation is used as a lightweight signal to identify tasks whose high-level utilization patterns do not peak simultaneously. As shown in later subsections, this coarse-grained signal is sufficient to guide consolidation decisions that reduce power consumption without significantly degrading JCT. Since the correlation analysis above was performed on 600 tasks, it is essential to quantify the computational overhead of computing correlation coefficients, particularly as the sample size increases. To evaluate this cost, we measure the time required to compute the Pearson correlation for vectors of different lengths. As shown in Table 3.1, the computation of correlations incurs minimal overhead. Even with 1 million samples, the runtime is under 13 ms. This confirms that

correlation analysis imposes negligible overhead and can be performed online without affecting scheduling responsiveness.

Table 3.1: Overhead for Correlation Computation

Samples	**1K**	**10K**	**100K**	**500K**	**1000K**
Time (ms)	0.295	0.325	1.325	6.469	12.999

3.4.3 Consolidation Analysis

Several works discuss multiplexing approaches, such as GPUlet [98], IADeep [99], and Orion [100], which explore task consolidation on the same GPU. However, *all of these focus on inference tasks*. However, a recent study analyzes GPU workload consolidation for deep learning training tasks on NVIDIA GPUs [101]. It shows that for small and medium-sized training jobs that underutilize a GPU, running multiple jobs concurrently (e.g., via MPS or MIG) can significantly increase overall training throughput, in some cases by up to 3×. To determine whether it is feasible to schedule multiple DNN workloads on the same GPU, we analyze real-world traces (the PAI trace [7]). This analysis serves two main purposes: first, it assesses whether GPUs have enough DRAM capacity to accommodate the tasks; second, it verifies that tasks can be scheduled on the same GPU without exceeding that DRAM capacity. This step is crucial because the correlation analysis focuses on scheduling multiple tasks or layers to a single GPU.

The PAI trace from Alibaba [7] captures a mix of ML training and inference jobs executed on a large-scale cluster comprising over 6,500 GPUs during July and August 2020. Each job in the trace is associated with a unique identifier, start time, duration,

requested GPU memory, requested GPU utilization, and job status. We preprocess the trace by applying the following filters: (1) we exclude jobs marked as failed, (2) we remove jobs with zero GPU memory or GPU utilization, and (3) we filter out jobs with execution times shorter than 20 seconds, as these typically correspond to inference workloads [54].

Figure 3.2(b) presents the CDF of DRAM usage, showing that most workloads consume significantly less than 16 GB, as indicated by the steep increase in the CDF. Additionally, most workloads require less than 32 GB. Therefore, scheduling multiple workloads on the same GPU is feasible when the aggregate working sets (including short-term peaks) fit within a GPU with at least 32 GB of DRAM, with sufficient headroom to avoid allocator fragmentation and framework/runtime reservations.

Observation 2: GPU memory consumption is fragmented, providing an opportunity to consolidate tasks.

Consolidating multiple tasks can significantly impact the JCT of each task [54, 68, 75, 100, 102, 103]. Hence, we need to ensure that consolidation does not exceed the task's SLA [76]. For example, Orion [100] suggests that consolidating a compute-oriented task with a memory-bound task reduces contention. Similarly, Yeung et al. [75] propose proactively predicting GPU utilization before scheduling so that jobs are consolidated only when their combined utilization is unlikely to exceed GPU capacity. Motivated by our empirical analysis of GPU utilization and memory usage in production traces, we design `CATS` to favor consolidation of tasks with complementary utilization patterns. Specifically, `CATS` consolidates tasks only when (1) the aggregate DRAM footprint of the consolidated tasks fits within the capacity of a single GPU, and (2) their GPU utilization patterns are complementary such that their resource demands do not peak simultaneously. By consolidating tasks with non-overlapping

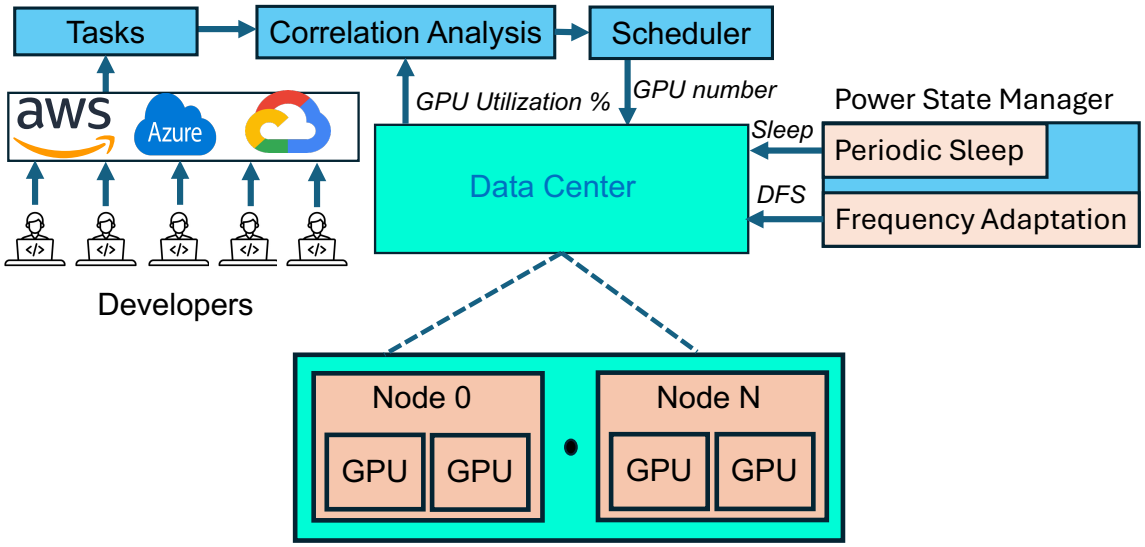


Figure 3.3: Architecture of the proposed CATS framework. The framework comprises correlation analysis, a scheduler, and a power-state manager that performs periodic sleep and frequency adaptation.

utilization phases, CATS mitigates resource contention and avoids excessive increases in JCT. In the next subsection, we present our CATS framework.

3.4.4 CATS Framework

The design of our proposed CATS framework is illustrated in Figure 3.3. CATS includes a *correlation analysis* module that performs correlation analysis of task GPU utilization. It has a *scheduler* that dispatches tasks to datacenter GPUs. The *power state manager* runs periodically and consists of two modules: a *sleep module*, which puts unused GPUs and servers to sleep, and a *frequency adaptation module*, which dynamically adjusts the frequency of active GPUs.

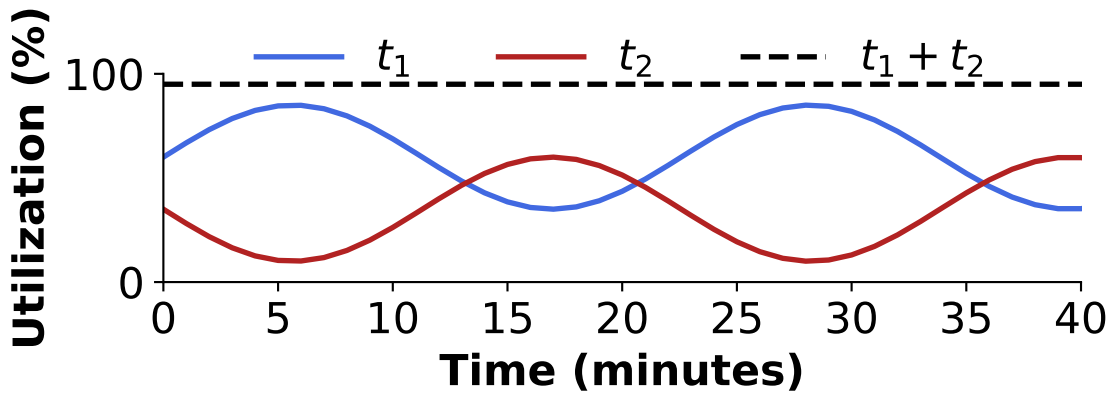
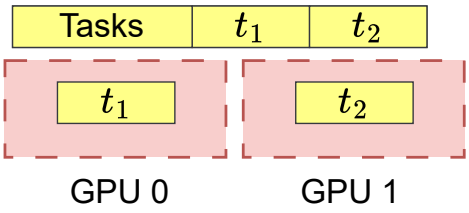


Figure 3.4: The two tasks are negatively correlated, and the aggregated utilization is less than 100%.

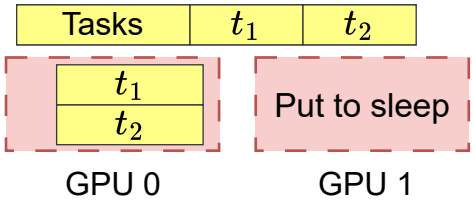
Our framework has several benefits. (1) It can integrate services based on ML, such as a resource manager aware of burstiness [104] and a network-aware job scheduler [105]. (2) It offers high usability: The proposed scheduling approach can be integrated with systems such as Kubernetes or YARN using existing plugin interfaces or custom schedulers. The algorithm relies solely on task metadata and lightweight computations, allowing it to operate efficiently within control loops. Users do not need to provide extra information or specifications. (3) It incurs low overhead: computation for correlation takes only milliseconds, as discussed in Section 3.4.2, which is negligible for DL workloads.

We now illustrate a simple example to demonstrate how `CATS` works. In this example, we consider two training tasks: t_1 with 12 GB of memory and t_2 with 10 GB of memory from the Philly trace [8]. The average GPU utilization of t_1 and t_2 is 52.23% and 42.7%, respectively. The utilization patterns of these tasks are shown in Figure 3.4. In this example, we consider our testbed setup with 2 GPUs, each

having 32 GB of memory. Figure 3.5(a) shows the default approach [10] [63], where each task is assigned to a separate GPU. We now compare it with CATS. For CATS at time 0, when t_1 arrives, we assign it to GPU 0. At time 1 second, task t_2 arrives. We compute the correlation coefficient ($\rho(t_1, t_2)$ using Equation (3.15)) between the task already running on GPU 0 and the incoming task t_2, which is -1. Since the correlation coefficient is negative, and their total memory usage (22 GB) is less than the GPU's DRAM capacity (32 GB), we consolidate t_1 and t_2 on GPU 0 as shown in Figure 3.5(b).



(a) Default



(a) CATS

Figure 3.5: An example of ML task consolidation by CATS. (a) shows the initial task assignment. (b) shows the aggregated task consolidation after applying CATS. We put the unused GPU (GPU 1), to sleep to reduce power consumption.

While distributing each task to its individual GPU results in the shortest JCT for all tasks, the JCT for our proposed method, CATS, is only slightly longer, as we utilize correlation and the tasks do not contend for resources. For the CATS GPU 1 can be put to sleep to reduce idle power consumption. Additionally, if a server is equipped with only one GPU, as in Alibaba's infrastructure [50], the server can be shut down or put to sleep when the GPU is no longer in use, reducing idle server power consumption. If a server contains two or more GPUs, it can be transitioned

to a sleep state only after all GPUs on that server have been put into sleep. We discuss how this is enforced in Section 3.4.5. In addition, to reduce dynamic power consumption for `CATS`, GPU 0 can adjust its operating frequency via DFS at runtime as discussed in Section 3.4.5.

3.4.5 Correlation-Aware Consolidation Algorithm

We introduced in Section 3.3 that using mathematical tools to solve the optimal task consolidation is infeasible mainly due to its high computational complexity, as the task consolidation problem is *NP-hard*. We now propose a lightweight heuristic algorithm that achieves correlation-aware consolidation with low computational overhead. The algorithm we designed is based on the bin-packing principle. `CATS` comprises three key components: correlation analysis (Algorithm 2), scheduler (Algorithm 3), and power state manager (Algorithm 4 and Algorithm 5). We now describe each in detail.

Correlation Analysis. Algorithm 2 performs correlation analysis for each incoming task t_l. In lines $1-4$ it begins by initializing an empty buffer $\mathcal{B}$ and iterates over all servers $i \in [1, N]$ and their GPUs G_{ij}, where $j \in [1, w_i]$. If GPU G_{ij} has sufficient memory available (that is, $d_l < c_{ij}^{\text{mem}}$), we compute two metrics (i) the correlation coefficient (δ_g) between the tasks running on the GPU G_{ij} and the task t_l on line 6, (ii) the absolute difference in the GPU utilization (δ_μ) between the incoming tasks and existing tasks running on the GPU on line 7. Before computing δ_g, the utilization values are truncated to the length of the shorter one since the lengths must be the same to compute the correlation as stated in Equation (3.15).

Algorithm 2 Correlation Analysis

Upon arrival of a task $t_l \in L$

Input: Task memory d_l, task utilization $u(t_l)$, GPU utilizations $u(G_{ij})$, thresholds β, α, servers $i \in N$, trackers $\mathcal{A}, \mathcal{S}$

Initialize buffer $\mathcal{B} \leftarrow \emptyset$
while true **do**
    for each server $i \in [1, N]$ **do**
        for each GPU $G_{ij},\ j \in [1, w_i]$ **do**
            if $d_l < c_{ij}^{\text{mem}}$ **then**
                $\delta_g \leftarrow \rho(u(G_{ij}), u(t_l))$
                $\delta_\mu \leftarrow \|u(G_{ij}) - u(t_l)\|$
                $\mathcal{B} \leftarrow \mathcal{B} \cup \{(i, j, \delta_\mu, \delta_g)\}$
                if $G_{ij} \notin \mathcal{A}$ **then**
                    $\mathcal{A} \leftarrow \mathcal{A} \cup \{G_{ij}\}$
                end if
                if $i \notin \mathcal{S}$ **then**
                    $\mathcal{S} \leftarrow \mathcal{S} \cup \{i\}$
                end if
            end if
        end for
    end for
    if $\mathcal{B}$ is not empty **then**
        Scheduler$(\mathcal{S}, \mathcal{A}, \mathcal{B}, t_l)$
        break
    end if
end while

We perform correlation analysis immediately at job arrival (submission time). We do not require a full initial run; instead, we obtain the utilization trace using existing prediction methods for recurring tasks or, at startup, for new tasks via short-term profiling, as detailed in Section 3.4.1. We then compute correlations between the arriving task and the currently running tasks and use these correlations to select the target GPU/server. We use absolute utilization as an additional threshold because occasional capacity exceedance is acceptable. If the combined resource demand of two jobs temporarily exceeds capacity, it may result in a minor delay in execution time, but it will not cause failure or instability. It is beneficial to consolidate tasks with the

largest absolute difference in their GPU utilization [106] to reduce contention during consolidation. We add the GPU used, the server used, the correlation coefficient, and the average utilization to the *buffer*. If G_{ij} is not already present in the active GPU tracker $\mathcal{A}$, it is added; similarly, if server i is not present in the server tracker $\mathcal{S}$, it is also added. If the buffer $\mathcal{B}$ is not empty, the scheduler is invoked (line 19) to evaluate each candidate GPU in $\mathcal{B}$ to determine the most suitable placement for task t_l.

Scheduler. Algorithm 3 schedules the incoming task using the candidates in $\mathcal{B}$. In lines $1, 2$, the scheduler initializes the candidate list $\mathcal{C}$ and sets the weights (λ_1 and λ_2). Currently, we assign equal weights because the scheduler allows it to balance two objectives: maximizing the difference in aggregate utilization (δ_μ) and minimizing correlation (δ_g). Since oversubscribing GPUs can affect JCT [75], we jointly select the candidate with the largest absolute difference in utilization and the lowest correlation to reduce JCT. In lines $3 - 7$, the scheduler computes the weighted score and stores it in a temporary buffer $\mathcal{C}$. The scheduler then sorts the buffer $\mathcal{C}$ by s and schedules the task to the chosen GPU with the lowest score, as shown in lines $(8, 9)$.

Algorithm 3 Scheduler

Require: $\mathcal{B}$: candidate GPU tuples, $\mathcal{S}$: server tracker, $\mathcal{A}$: GPU tracker, incoming task t_l
Ensure: $\mathcal{K}$: selected GPU
1: $\mathcal{C} \leftarrow \emptyset$
2: Initialize weights (λ_1, λ_2)
3: **for** each $(i, j, \delta_\mu, \delta_g) \in \mathcal{B}$ **do**
4: score $\leftarrow \lambda_1 \delta_g - \lambda_2 \delta_\mu$
5: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(i, j, \text{score})\}$
6: **end for**
7: Sort $\mathcal{C}$ in ascending order of score
8: $(\mathcal{K}, \text{gpu}) \leftarrow$ GPU associated with the smallest score in $\mathcal{C}$
9: Dispatch task t_l to GPU $\mathcal{K}$

Power State Manager. The power state manager in `CATS` minimizes the average power consumption over the upcoming scheduling window, thereby reducing the total energy expenditure within that interval. Because the JCT is constrained by a service-level agreement (SLA), as specified in Algorithm 5, a reduction in average power consumption translates directly into proportionally lower energy usage. The power state manager comprises two components: (1) the periodic sleep module and (2) the frequency adaptation modules. We describe each module in detail below.

The *periodic sleep* module is described in Algorithm 4. This module runs on a background thread and executes at regular intervals of length τ. In this chapter, we set τ to 100 ms to match the frequency adjustment overhead of `nvidia-smi` [56]. Synchronizing the sleep interval with the frequency adjustment overhead ensures that both mechanisms operate efficiently without introducing additional delays. The periodic sleep module in Algorithm 4 scans all servers (line 1). For each server, it checks whether all GPUs are either idle or have no active tasks. If they are idling, it puts them to sleep (lines $4-7$). After scanning all GPUs, if none are active, the entire server is put to sleep (lines $10-12$).

The *Frequency Adaptation* module, stated in Algorithm 5, runs in the background at regular intervals of length τ and performs DFS. As discussed in Section 3.3, lowering GPU frequency reduces power consumption but can increase job completion time (JCT). Accordingly, our goal is to balance performance and power efficiency while keeping execution times within acceptable bounds. In Algorithm 5, the minimum and maximum frequencies ($f_{\min}$ and $f_{\max}$) of the GPU and the SLA factor (γ) for all tasks are the inputs provided when the frequency adaptation module is launched. For each active GPU g, the algorithm iterates over all tasks $\mathcal{T}_g$ currently running on

Algorithm 4 Periodic Sleep

Require: Runs every interval τ; servers $i \in [1, N]$ with w_i GPUs each; active set $\mathcal{S}$

```
1: for i = 1 to N do
2:     server_active ← false
3:     for j = 1 to w_i do
4:         if G_ij has running tasks then
5:             server_active ← true
6:         else
7:             sleep G_ij
8:         end if
9:     end for
10:    if server_active = false then
11:        sleep server i
12:        S ← S \ {i}
13:    end if
14: end for
```

it. In line 8, for every task $t \in \mathcal{T}_g$, we estimate the prediction time of the task based on the model from Section 3.3. If the predicted time exceeds the maximum allowed time (line 10), a violation is recorded, and *violation* is set to true. If the predicted time is within 5% of the limit (lines $12 - 13$), then *slack* is set to false. After all tasks are checked, the GPU frequency is updated (lines $16 - 19$). If any task violates its bound, the frequency increases by Δf but is capped at f_{max}. If no violations occur and all tasks have slack, the frequency is reduced by Δf but not below f_{min}. If tasks are close to the limit (within 5% of e_t^{max}), the frequency remains unchanged to avoid oscillations.

Complexity Analysis. The time complexity of *Correlation Analysis (Algorithm 2)*, for each incoming task, is that the algorithm scans every GPU in the server. For each GPU, it computes the correlation coefficient and the difference in utilization, both of which require O(L) operations on the utilization vectors of length L. Thus, the time complexity per invocation is $O(G \cdot L)$, where G is the total number of GPUs.

Algorithm 5 Frequency Adaptation

Require: Runs every interval τ; GPU set $\mathcal{K}$; SLA factor γ; frequency bounds $f_{\min}, f_{\max}$

for each $g \in \mathcal{K}$ **do**
  $f_g \leftarrow$ current frequency of GPU g
  $\beta \leftarrow$ curve-fitting constant
  $\mathcal{T}_g \leftarrow$ tasks running on GPU g
  violation $\leftarrow$ **false**
  slack $\leftarrow$ **true**
  for each $t \in \mathcal{T}_g$ **do**
    $e_t^{\text{pred}} \leftarrow e_{\min,t} \cdot \left(\frac{f_{\max}}{f_g}\right)^{\beta}$
    $e_t^{\max} \leftarrow \gamma \cdot e_{\min,t}$
    if $e_t^{\text{pred}} > e_t^{\max}$ **then**
      violation $\leftarrow$ **true**
    else if $e_t^{\text{pred}} \geq 0.95 \cdot e_t^{\max}$ **then**
      slack $\leftarrow$ **false**
    end if
  end for
  if violation **then**
    $f_g \leftarrow \min(f_{\max},\ f_g + \Delta f)$
  else if slack **then**
    $f_g \leftarrow \max(f_{\min},\ f_g - \Delta f)$
  end if
end for

For the *Scheduler (Algorithm 3).* the scheduler evaluates each candidate GPU in the buffer $\mathcal{B}$ and assigns it a score,which costs $O(|\mathcal{B}|)$, and then sorts the scores, which costs $O(|\mathcal{B}| \log |\mathcal{B}|)$. Therefore, the total time complexity is $O(|\mathcal{B}| \log |\mathcal{B}|)$ which is at most $O(G \log G)$. The *Periodic Sleep (Algorithm 4)* procedure scans every server and every GPU once per interval. Each check is constant time, so the total time complexity is $O(G)$. For the *Frequency Adaptation (Algorithm 5).* This module iterates over all active GPUs and over all tasks running on them. Let $|\mathcal{K}|$ be the number of active GPUs and T the total number of active tasks. The total time complexity per invocation is $O(|\mathcal{K}| + T)$.

3.4.6 Discussion

Scalability. To address scalability concerns in large data centers with thousands of GPUs, `CATS` can be deployed at three levels of granularity: *data center level*, *cluster level*, and *server level*. In *Data Center Level* managing all GPUs through a single centralized controller can lead to significant overhead. To mitigate this, correlation analysis can be performed less frequently, such as every 10 minutes, and simpler task allocation methods like first-fit can be utilized. This approach reduces computational complexity while still enabling power optimization across the entire data center. For *Cluster Level* we divide the data center into smaller clusters, each managed by its own `CATS` controller, which lowers the computational burden compared to a fully centralized approach. Within each cluster, correlation analysis can be conducted more regularly, for example, every 5 minutes, allowing for more responsive task dispatching and frequency scaling. *Server Level* has the highest level of granularity. Here `CATS` can be implemented on individual servers. Each server's controller manages only the GPUs within that server, enabling real-time task scheduling and frequency adjustments with minimal overhead.

Task Migration. `CATS` does not perform Dynamic migration because it's expensive for ML training and requires checkpointing, transferring, and restoring GPU-resident model and optimizer state, which can take seconds to minutes and scales with model size. Migration also disrupts placement locality in distributed training and can affect performance [107].

Multidimensional Correlation. `CATS` can be extended to a multidimensional resource model (e.g., Memory utilization, HBM bandwidth, interconnect) by representing each job as a resource vector and performing per-dimension correlation checks

subject to capacity constraints. However, such an extension scales linearly with the number of dimensions, preserving lightweight execution within the sub-millisecond window. Currently, `CATS` conducts correlation analysis only for GPU utilization and considers other resource demands, such as memory capacity, as hard constraints.

Extension to LLM workloads. Although our evaluation focuses on conventional DNN training jobs, `CATS` naturally extends to large language model (LLM) workloads. Compared to traditional DNN training, LLM execution exhibits more structured and predictable phases, such as attention computation and KV-cache updates, which often lead to bursty and memory-bound behavior. Prior work shows that these phases can be accurately predicted [37]. Because `CATS` identifies tasks whose coarse-grained utilization patterns do not peak simultaneously, these predictable execution phases make LLM workloads particularly well suited for consolidation. For example, a compute-intensive prompt-processing task can be consolidated with a memory-bound token-generation task when their utilization phases complement each other, and their combined memory footprint fits within the GPU capacity.

3.5 Experimental Setup

Hardware Testbed. Our testbed consists of a single node-server with six Nvidia Volta-V100 GPUs. Table 3.2 describes the DRAM consumption of the GPUs on the server. The server runs on Ubuntu 20.04 LTS and is powered by a 1.7 GHz CPU, and the GPUs operate at frequencies between 135 MHz and 1350 MHz. Task scheduling is performed online using PyTorch 1.8.1. To enable spatial sharing, the CUDA Multiprocessing Service (MPS) is used. The MPS thread percentage is set to 100% to

ensure complete GPU provisioning. In all experiments, we set the correlation threshold to 0 and the utilization threshold to 100%. We measure GPU power consumption and utilization using the `nvidia-smi` interface.

Table 3.2: GPU Configuration in the Hardware Testbed

GPU	**I**	**II**	**III**	**IV**	**V**	**VI**
GPU DRAM Capacity (GB)	16	16	32	32	16	32

Workloads. The workloads for the experiments are as follows: t_1 corresponds to DenseNet with a batch size of 50, t_2 is GoogleNet with a batch size of 30, t_3 is ConvNeXt [108] with a batch size of 50, t_4 is Vision Transformer (ViT) with a batch size of 60, t_5 is Swin Transformer [74] with a batch size of 40, t_6 is ResNet [74] with a batch size of 30, t_7 is MobileNet [71] with a batch size of 50, and finally t_8 is SqueezeNet [109] with a batch size of 60. The workloads are derived from the PAI trace and were selected to (i) reflect realistic GPU utilization levels observed in production traces and (ii) ensure diverse memory footprints that enable meaningful consolidation under DRAM constraints [54] [51].

Baselines. The baselines used for comparison are `(1) Hotcloud` and `(2) First`. `Hotcloud` calculates the sum of the average utilization of the existing and incoming tasks on the GPU. If the combined average utilization of the two is below the utilization threshold, `Hotcloud` schedules them on the same GPU. The GPU utilization prediction follows the approach suggested by Yeung et al. [75]. `First` computes the sum of GPU utilization during the first second of the incoming task and adds it to

the aggregated GPU utilization of tasks already running on the GPU. If their total utilization is less than the threshold (q), it schedules the tasks to the same GPU.

Trace-driven Simulation. To address the limitations of our hardware testbed, we developed a Python simulator to evaluate `CATS` on a large scale using real-world traces. The simulator is built from the open-source code of Gavel [63] and GPUColo [54], and is designed to emulate a heterogeneous data center. The simulator features several key components: a simulated clock module, a scheduler, a server, a GPU manager, and a power module. Before the simulation begins, all jobs are sorted by their release times in ascending order. Jobs are then assigned to GPUs using the scheduling strategy specific to each baseline; for each baseline, we apply its corresponding scheduling solution as described in the previous paragraph. The simulator also includes a power management component that continuously monitors the power of active GPUs and servers. The power manager records power consumption over time, applies consolidation policies such as dynamic frequency scaling or periodic sleep, and ensures that idle resources do not consume unnecessary power.

3.6 Hardware Evaluation

We evaluate `CATS` on our GPU testbed using workloads sampled from production traces. We use 14 machine learning (ML) tasks, sampled from the PAI trace [7], selected based on their GPU utilization and memory requirements. To reflect the bursty arrival patterns observed in the PAI trace, we structure the experiment into two stages, each consisting of a group of tasks released within a short time window, as summarized in Table 3.3. Stage 1 comprises eight tasks t_1 to t_8 and stage 2 comprises six tasks t_9 to t_{14}. Initially, four tasks (t_1–t_4) are released sequentially at 0, 1, 2, and

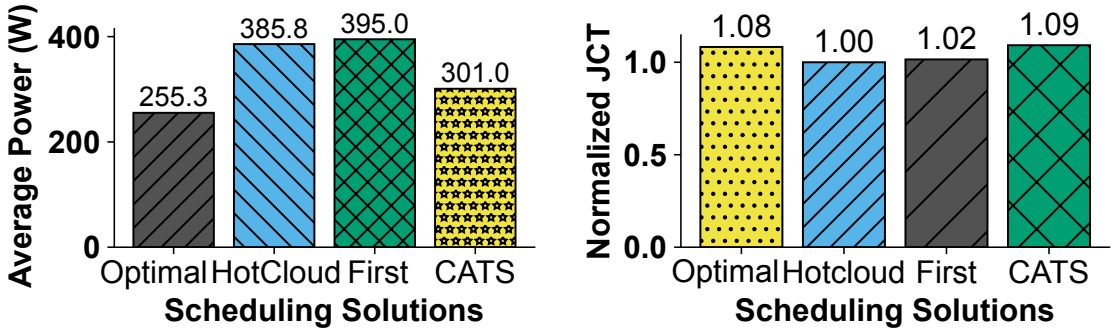
3 seconds. Four additional tasks (t_5–t_8) are then launched at $20 - 23$ seconds. In Stage 2 at time 500 seconds, four tasks (t_9, t_{10}, t_{11}, and t_{12}) are released. Two more tasks (t_{13} and t_{14}) are released at 520 and 522 seconds. The workloads described in Section 3.5 are reused across the tasks. Specifically, tasks t_1 to t_8 use the workloads of Section 3.5, while tasks t_9 to t_{14} replicate the same workloads as t_1 to t_8.

Table 3.3: Task start time, GPU utilization (%), and memory consumption (GB).

Task	Start time	GPU util. (%)	Mem (GB)
t_1	0	45.84	6.28
t_2	1	44.44	6.30
t_3	2	40.00	2.20
t_4	3	46.00	1.80
t_5	20	49.00	3.90
t_6	21	47.00	1.60
t_7	22	45.00	2.20
t_8	23	44.00	3.50
t_9	500	45.84	6.28
t_{10}	500	44.44	6.30
t_{11}	500	40.00	2.20
t_{12}	500	46.00	1.80
t_{13}	520	49.00	3.90
t_{14}	522	47.00	1.60

We design this experiment to evaluate the robustness of CATS under model prediction error. The objective is to demonstrate that the correlation-aware scheduler remains effective even when task utilization predictions are noisy or partially inaccurate. To achieve this, we construct a synthetic GPU utilization trace for the selected 14 tasks. For each task, we sample from a Gaussian distribution with mean (μ) and

variance (σ^2) matching those observed in real ML training workloads. To emulate prediction uncertainty, we add additive white noise to the generated utilization profiles, introducing controlled perturbations to the predicted task behavior.



(a) Power

(b) Normalized JCT

Figure 3.6: (a) The `optimal` solution has the smallest power consumption. (b) The normalized JCT is similar across all solutions, and the `optimal` solution remains within the SLA threshold.

In the first step, we measure power consumption across all stages and calculate the mean power consumption for each solution, as illustrated in Figure 3.6(a). The figure indicates that the `optimal` solution has the lowest power consumption, averaging 255.3 W. In comparison, `CATS`, `HotCloud`, and `First` consume 301 W, 385.8 W, and 395 W, respectively. The `optimal` solution achieves this lower power consumption by utilizing the fewest GPUs. Although the `optimal` solution offers an upper bound for comparison, its high computational overhead limits its scalability. Therefore, we only present results for the `optimal` solution in our hardware testbed experiments and exclude it from larger-scale simulations.

`CATS` achieves significantly lower power consumption compared to HotCloud and `First`. This is due to `CATS` consolidating several tasks onto fewer GPUs, placing

unused GPUs into a low-power sleep state, and applying frequency scaling to reduce GPU dynamic power consumption. Reducing power consumption is typically achieved by decreasing the number of GPUs, which may increase job completion time (JCT). To assess whether this increase violates the Service Level Agreement (SLA), we compute the normalized JCT for all tasks and normalize it with respect to `CATS`. We set the SLA threshold to 20% of the fastest baseline as mentioned in Section 3.3. Figure 3.6(b) presents the normalized JCT for all techniques. We observe that `CATS` maintains its JCT within the SLA. This is mainly because `CATS` consolidates tasks with negative correlations, thereby reducing resource contention.

3.7 Simulation Evaluation

In this section, we conduct simulation experiments across all baselines and `CATS` using the simulator mentioned in Section 3.5. As discussed in Section 3.3, we do not perform the simulation experiment for the `optimal` solution because determining the optimal consolidation is computationally expensive.

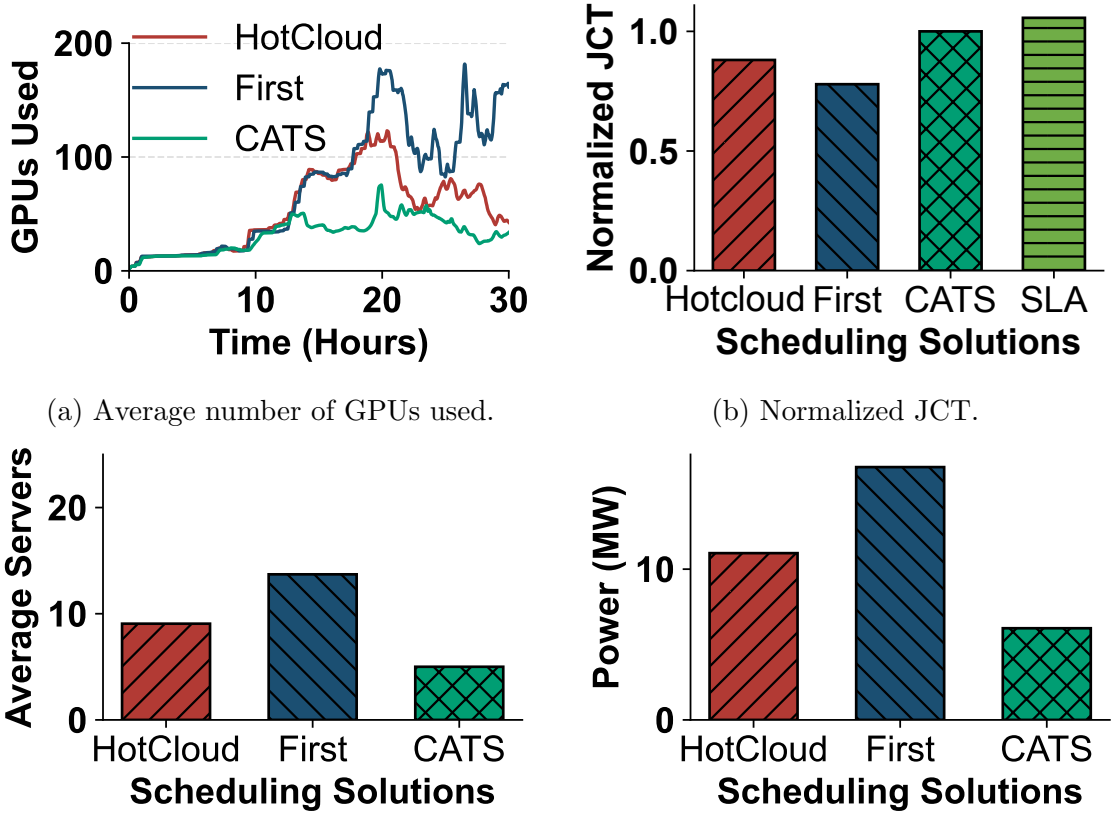
We perform simulation experiments on the two traces (*PAI* and *Philly*) mentioned in Section 3.5. To ensure a fair comparison at the start of the simulation, all GPUs and servers are put to sleep and activated only when a task is dispatched. Whenever the GPU or a server is idle, the *periodic* sleep module is activated and puts unused GPUs and servers to sleep for all the solutions. In all the experiments, each server is equipped with 8 GPUs.

PAI Trace. We choose 5000 tasks from the *PAI* trace to perform the simulation. Figure 3.7(a) shows the average number of GPUs used throughout the experiment. `CATS` requires fewer GPUs because it consolidates more tasks per GPU than `HotCloud`.

On average, CATS uses 37.32 GPUs, First uses 92.94 GPUs, and HotCloud uses 61.94 GPUs. CATS uses 39.75% fewer GPUs than HotCloud. The reduction in the number of GPUs must not exceed the SLA. As shown in Figure 3.7(b), CATS exhibits a slightly higher JCT than the baseline approaches, HotCloud and First, which have JCT values of 0.87 and 0.77, respectively. In contrast, CATS's JCT is 1, which remains within the due time threshold of $1.2 \times 0.87 = 1.044$, where 0.87 is HotCloud's JCT. CATS reduces resource contention by consolidating tasks with negative correlations. CATS also uses fewer servers compared to other solutions, as shown in Figure 3.7(c). CATS has the lowest power consumption because it puts unused servers and GPUs to sleep to reduce idle power, and it applies DFS to reduce dynamic power. CATS's power consumption is 6.08 MW, which is 63.59% less than First's 16.7 MW as shown in Figure 3.7(d).

Philly Trace. We choose 2000 tasks from the *Philly* trace to perform the simulation. Figure 3.8(a) shows the average number of GPUs used throughout the experiment. CATS requires fewer GPUs because it consolidates more tasks per GPU than HotCloud.

The reduction in the number of GPUs must not exceed the SLA. As shown in Figure 3.8(b), CATS exhibits a slightly higher JCT than the baseline approaches, HotCloud and First. Normalized to CATS, the JCT values are 0.857 for HotCloud, 0.974 for First, and 1.0 for CATS. CATS's normalized JCT remains within the due time threshold of $1.2 \times 0.857 = 1.028$, where 0.857 corresponds to HotCloud's normalized JCT. CATS reduces resource contention by consolidating tasks with negative correlations. CATS also uses fewer servers compared to other solutions as shown in Figure 3.8(c). CATS has the lowest power consumption because it puts unused servers



(a) Average number of GPUs used.

(b) Normalized JCT.

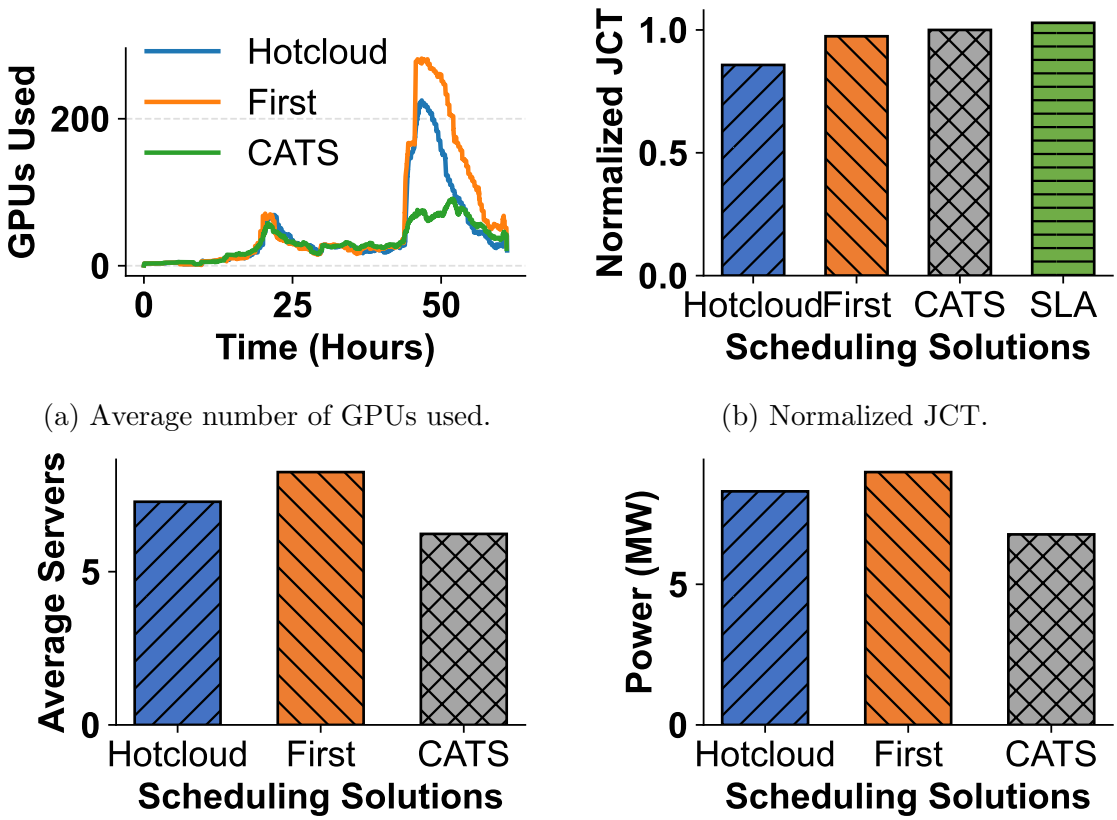
(c) Average number of servers used.

(d) Average power consumption.

Figure 3.7: Resource utilization, performance, and power consumption results for the PAI trace simulation. (a) `CATS` uses the fewest GPUs. (b) The normalized job completion time (JCT) of `CATS` remains within the SLA. (c) `CATS` uses the fewest servers. (d) `CATS` achieves the lowest average power consumption.

and GPUs to sleep to reduce idle power and applies DFS to reduce dynamic power consumption. The average power consumption of `CATS` is 6.77 MW, compared to 8.30 MW for `HotCloud` and 8.99 MW for `First`. This corresponds to a reduction of 18.4% compared to `HotCloud` and 24.7% compared to `First` as shown in Figure 3.8(d).

Next, we compare `CATS` with other task consolidation solutions that perform consolidation based on resource attributes such as memory consumption. These approaches are orthogonal to utilization-aware GPU scheduling techniques such as



(a) Average number of GPUs used.

(b) Normalized JCT.

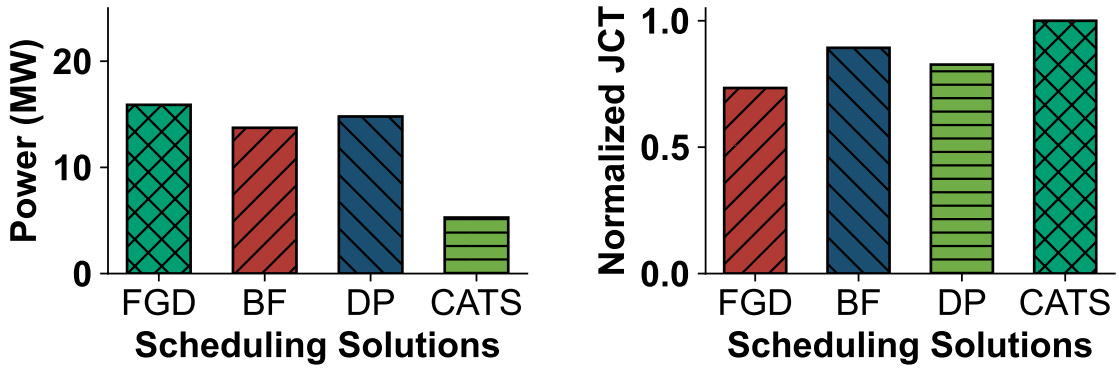
(c) Average number of servers used.

(d) Average power consumption.

Figure 3.8: Resource utilization, performance, and power consumption results for the Philly trace simulation. (a) `CATS` uses the fewest GPUs. (b) The normalized job completion time (JCT) of `CATS` remains within the SLA. (c) `CATS` uses the fewest servers. (d) `CATS` achieves the lowest average power consumption.

`CATS`, but they influence how tasks are colocated on shared resources and can therefore be used as baselines for comparison. The consolidation solutions we consider are `Best-Fit (BF)` [110], `Dot-Product (DP)` [80], and `FGD` [50]. `Best-Fit (BF)` assigns a task to the node that results in the least remaining resources after placement, computed as a weighted sum across resource dimensions. `Dot-Product (DP)` selects the node that minimizes the dot product between the task's resource demand vector

and the node's available resource vector. FGD mitigates GPU fragmentation by placing tasks along the steepest descent of fragmentation, thereby improving allocation efficiency.



(a) Power consumption.

(b) Normalized JCT.

Figure 3.9: (a) CATS achieves the lowest power consumption. (b) Normalized JCT for all the solutions.

Figure 3.9(a) shows the average power consumption across all solutions. Among all the solutions, CATS achieves the lowest power consumption, using approximately 5.28 kW on average, while FGD, BF, and DP consume 15.89 kW, 13.73 kW, and 14.79 kW, respectively. CATS reduces power consumption by up to 61.6%. This reduction is primarily due to two factors. First, CATS uses fewer GPUs by consolidating tasks with negative performance correlations. Second, CATS applies DFS to further reduce GPU dynamic power consumption. Figure 3.9(b) presents the normalized job completion time (JCT). The other solutions have a better JCT because these solutions are optimized for the shortest makespan, without accounting for power consumption.

3.8 Conclusion and Future Work

In this chapter, we present `CATS`, a novel power-optimization framework for GPU data centers. In sharp contrast to existing work that assumes GPU utilization for each ML task can be approximated as a constant, `CATS` features correlation-aware scheduling to analyze GPU utilization patterns across ML tasks. By consolidating tasks with negatively correlated GPU utilization on the same GPUs, `CATS` reduces the number of active GPUs required, thereby lowering overall power consumption without significantly increasing the job completion time (JCT) of the ML tasks. We formulate correlation-aware ML task consolidation as a constrained optimization problem with a costly optimal solution. We then design a light-weight heuristic algorithm that is practical for real data centers. Our experiments, conducted through extensive simulations, demonstrate that `CATS` reduces power consumption by up to 24.7% relative to the baselines while maintaining comparable JCTs.

Chapter 4: Conclusion and Future Work

The rapid growth of AI/ML workloads has made GPU cost and power efficiency critical challenges in modern data centers. In this work, we showed that ML workloads often exhibit weak or negative utilization correlations, creating opportunities for more efficient resource sharing. To leverage this insight, we proposed two correlation-aware solutions: `CorrGPU`, which minimizes GPU provisioning cost (CapEx), and `CATS`, which reduces runtime power consumption (OpEx). Our experimental results demonstrate significant improvements, achieving up to **20.88% reduction in CapEx** and up to **24.7% reduction in power consumption**. Overall, this work highlights that correlation-aware scheduling provides a practical and scalable approach to improving efficiency in GPU data centers.

As future work, we plan to extend these frameworks to support emerging **large language model (LLM) workloads**, which introduce new challenges, including dynamic sequence lengths, higher memory pressure, and heterogeneous serving patterns. Adapting correlation-aware scheduling to LLM workloads will further enhance efficiency and applicability in next-generation AI systems.

Bibliography

[1] Mauro Ribeiro et al. Mlaas: Machine learning as a service. In *ICMLA*, 2015.

[2] Martín Abadi et al. Tensorflow: Large-scale machine learning on heterogeneous distributed systems. *arXiv preprint arXiv:1603.04467*, 2016.

[3] Adam Paszke et al. Pytorch: An imperative style, high-performance deep learning library. *NeurIPS*, 2019.

[4] Yangqing Jia et al. Caffe: Convolutional architecture for fast feature embedding. In *Proceedings of the 22nd ACM international conference on Multimedia*, 2014.

[5] Kaiming He et al. Deep residual learning for image recognition. In *CVPR*, 2016.

[6] Ian Tenney et al. Bert rediscovers the classical nlp pipeline. *arXiv preprint arXiv:1905.05950*, 2019.

[7] Mengdi Wang, Chen Meng, Guoping Long, Chuan Wu, Jun Yang, Wei Lin, and Yangqing Jia. Characterizing deep learning training workloads on alibaba-pai. In *IEEE international symposium on workload characterization (IISWC)*, 2019.

[8] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. Analysis of large-scale multi-tenant gpu clusters for dnn training workloads. In *USENIX Annual Technical Conference (USENIX ATC)*, 2019.

[9] Zhisheng Ye, Wei Gao, Qinghao Hu, Peng Sun, Xiaolin Wang, Yingwei Luo, Tianwei Zhang, and Yonggang Wen. Deep learning workload scheduling in gpu datacenters: A survey. *ACM Computing Surveys*, 56(6), 2024.

[10] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Synergy: Resource sensitive dnn scheduling in multi-tenant clusters. *arXiv preprint arXiv:2110.06073*, 2021.

[11] Aditya Dhakal, Sameer G Kulkarni, and KK Ramakrishnan. Gslice: controlled spatial sharing of gpus for a scalable inference platform. In *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC)*, 2020.

[12] Daegun Yoon and Sangyoon Oh. Deft: Exploiting gradient norm difference between model layers for scalable gradient sparsification. In *Proceedings of the 52nd International Conference on Parallel Processing*, 2023.

[13] Yixin Bao, Yanghua Peng, and Chuan Wu. Deep learning-based job placement in distributed machine learning clusters. In *IEEE conference on computer communications (INFOCOM)*, 2019.

[14] Guoyu Chen and Xiaorui Wang. Performance optimization of machine learning inference under latency and server power constraints. In *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*, 2022.

[15] Srinivasan Subramaniyan and Xiaorui Wang. Opticpd: Optimization for the canonical polyadic decomposition algorithm on gpus. In *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 403–412, 2023.

[16] M. R. Ashuthosh, Santosh Krishna, Vishvas Sudarshan, Srinivasan Subramaniyan, and Madhura Purnaprajna. Mapparat: A resource constrained fpga-based accelerator for sparse-dense matrix multiplication. In *2022 35th International Conference on VLSI Design and 2022 21st International Conference on Embedded Systems (VLSID)*, pages 102–107, 2022.

[17] Srinivasan Subramaniyan, Oscar Ferraz, M. R. Ashuthosh, Santosh Krishna, Guohui Wang, Joseph R. Cavallaro, Vitor Silva, Gabriel Falcao, and Madhura Purnaprajna. Pushing the limits of energy efficiency for non-binary ldpc decoders on gpus and fpgas. In *2020 IEEE Workshop on Signal Processing Systems (SiPS)*, pages 1–6, 2020.

[18] Oscar Ferraz, Srinivasan Subramaniyan, Guohui Wang, Joseph R Cavallaro, Gabriel Falcao, and Madhura Purnaprajna. Gbit/s non-binary ldpc decoders: High-throughput using high-level specifications. In *2020 28th IEEE Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2020.

[19] Srinivasan Subramaniyan, Oscar Ferraz, MR Ashuthosh, Santosh Krishna, Guohui Wang, Joseph R Cavallaro, Vitor Silva, Gabriel Falcao, and Madhura Purnaprajna. Enabling high-level design strategies for high-throughput and low-power nb-ldpc decoders.

[20] M. Ji, B. Vemparala, P. Mageswaran, G. G. Knapik, D. M. Blakaj, E. C. Bourekas, E. Mendel, W. S. Marras, and S. Soghrati. Deep learning surrogate modeling for multiscale trabecular bone remodeling simulations. *Engineering with Computers*, 2026. Accepted for publication; in press.

[21] Balavignesh Vemparala, Mingshi Ji, Prasath Mageswaran, Gregory G Knapik, Khaled Dibs, Dukagjin M Blakaj, Eric C Bourekas, Ehud Mendel, William S Marras, and Soheil Soghrati. A computational multiscale framework for bone remodeling: Coupling apparent density evolution and microscale shape optimization. *International Journal for Numerical Methods in Biomedical Engineering*, 41(10):e70097, 2025.

[22] Balavignesh Vemparala, Ming Yang, and Soheil Soghrati. Deep learning-driven domain decomposition (dld3): A generalizable ai-driven framework for structural analysis. *Computer Methods in Applied Mechanics and Engineering*, 432, 2024.

[23] Balavignesh Vemparala Narayana Murthy. *Advanced Computational and Deep Learning Techniques for Modeling Materials with Complex Microstructures*. PhD thesis, The Ohio State University, 2024.

[24] Balavignesh Vemparala, Wadi H Imseeh, Salil Pai, Anand Nagarajan, Timothy Truster, and Soheil Soghrati. Automated reconstruction and conforming mesh generation for polycrystalline microstructures from imaging data. *Applied Sciences*, 14(1):407, 2024.

[25] Ryan P Connor, Balavignesh Vemparala, Reza Abedi, Giang Huynh, Soheil Soghrati, Chris T Feldmeier, and Kevin Lamb. Statistical homogenization of elastic and fracture properties of a sample selective laser melting material. *Applied Sciences*, 13(22):12408, 2023.

[26] VN Balavignesh, B Gundepudi, GR Sabareesh, and I Vamsi. Comparison of conventional method of fault determination with data-driven approach for ball bearings in a wind turbine gearbox. *International Journal of Performability Engineering*, 14(3):397, 2018.

[27] Akshar Shravan Chavan and Marco Brocanelli. Towards high-quality battery life for autonomous mobile robot fleets. In *2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, pages 61–70, 2022.

[28] Ze Li and Marco Brocanelli. An edge-based reinforcement learning approach for augmented reality apps in dynamic contexts. In *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing*, pages 1–16, 2025.

[29] Ze Li, Niloofar Didar, and Marco Brocanelli. Offloading-aware control of ai task allocation and virtual object quality in mar apps. *IEEE Transactions on Mobile Computing*, 2025.

[30] Tech's splurge on ai chips has companies in 'arms race' that's forcing more spending.
https://www.cnbc.com/2024/07/25/techs-splurge-on-ai-chips-has-meta-alphabet-tesla-in-arms-race.html.

[31] Nvidia. Nvidia h100 tensor core gpu. https://www.nvidia.com/en-us/data-center/h100/.

[32] Beth Kindig. Ai power consumption: Rapidly becoming mission-critical.
https://www.forbes.com/sites/bethkindig/2024/06/20/ai-power-consumption-rapidly-becoming-mission-critical/, 2024.

[33] Anton Shilov. Nvidia's h100 gpus will consume more power than some countries — each gpu consumes 700w of power, 3.5 million are expected to be sold in the coming year. https://www.tomshardware.com/, 2023.

[34] Yehia Arafa, Ammar ElWazir, Abdelrahman ElKanishy, Youssef Aly, Ayatelrahman Elsayed, Abdel-Hameed Badawy, Gopinath Chennupati, Stephan Eidenbenz, and Nandakishore Santhi. Verified instruction-level energy consumption measurement for nvidia gpus. In *Proceedings of the 17th ACM International Conference on Computing Frontiers*, 2020.

[35] Pratyush Patel, Zibo Gong, Syeda Rizvi, Esha Choukse, Pulkit Misra, Thomas Anderson, and Akshitha Sriraman. Towards improved power management in cloud gpus. *IEEE Computer Architecture Letters*, 2023.

[36] Seyed Morteza Nabavinejad, Sherief Reda, and Masoumeh Ebrahimi. Batchsizer: Power-performance trade-off for dnn inference. In *Proceedings of the 26th Asia and South Pacific Design Automation Conference*, 2021.

[37] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. Characterizing power management opportunities for llms in the cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 207–222, 2024.

[38] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K Iyer. Power-aware deep learning model serving with {μ-Serve}. In *USENIX Annual Technical Conference (USENIX ATC)*, 2024.

[39] Andreas Kosmas Kakolyris, Dimosthenis Masouros, Sotirios Xydis, and Dimitrios Soudris. Slo-aware gpu dvfs for energy-efficient llm inference serving. *IEEE Computer Architecture Letters*, 2024.

[40] Baolin Li, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. Clover: Toward sustainable ai with carbon-aware machine learning inference service. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.

[41] Munkyu Lee, Sihoon Seong, Minki Kang, Jihyuk Lee, Gap-Joo Na, In-Geol Chun, Dimitrios Nikolopoulos, and Cheol-Ho Hong. Parvagpu: Efficient spatial gpu sharing for large-scale dnn inference in cloud environments. In *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024.

[42] Farui Wang, Weizhe Zhang, Shichao Lai, Meng Hao, and Zheng Wang. Dynamic gpu energy optimization for machine learning training workloads. *IEEE Transactions on Parallel and Distributed Systems*, 33(11), 2021.

[43] Zhenheng Tang, Yuxin Wang, Qiang Wang, and Xiaowen Chu. The impact of gpu dvfs on the energy and performance of deep learning: An empirical study. In *Proceedings of the Tenth ACM International Conference on Future Energy Systems (e-Energy)*, 2019.

[44] Qinghao Hu, Peng Sun, Shengen Yan, Yonggang Wen, and Tianwei Zhang. Characterization and prediction of deep learning workloads in large-scale gpu datacenters. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2021.

[45] Xiaodong Wang, Yanjun Yao, Xiaorui Wang, Kefa Lu, and Qing Cao. Carpo: Correlation-aware power optimization in data center networks. In *Proceedings IEEE INFOCOM*, 2012.

[46] David Meisner, Brian T Gold, and Thomas F Wenisch. Powernap: eliminating server idle power. *ACM SIGARCH Computer Architecture News*, 2009.

[47] Akshat Verma, Gargi Dasgupta, Tapan Kumar Nayak, Pradipta De, and Ravi Kothari. Server workload analysis for power minimization using consolidation. In *Proceedings of the conference on USENIX Annual technical conference (USENIX ATC)*, 2009.

[48] Ching-Hsien Hsu, Kenn D Slagter, Shih-Chang Chen, and Yeh-Ching Chung. Optimizing energy consumption with task consolidation in clouds. *Elsevier Information Sciences*, 2014.

[49] Brandon Heller, Srinivasan Seetharaman, Priya Mahadevan, Yiannis Yiakoumis, Puneet Sharma, Sujata Banerjee, and Nick McKeown. Elastictree: Saving energy in data center networks. In *NSDI*, 2010.

[50] Qizhen Weng, Lingyun Yang, Yinghao Yu, Wei Wang, Xiaochuan Tang, Guodong Yang, and Liping Zhang. Beware of fragmentation: Scheduling gpu-sharing workloads with fragmentation gradient descent. In *USENIX Annual Technical Conference (USENIX ATC)*, 2023.

[51] Srinivasan Subramaniyan and Xiaorui Wang. Exploiting ml task correlation in the minimization of capital expense for gpu data centers. In *2025 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, pages 1–8. IEEE, 2025.

[52] Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. Pipeswitch: Fast pipelined context switching for deep learning applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[53] Srinivasan Subramaniyan and Xiaorui Wang. Cats: Correlation-aware task scheduling for gpu power optimization in ai data centers. In *Proceedings of the 2026 International Conference on Supercomputing (ICS '26)*. ACM, July 2026.

[54] Guoyu Chen, Srinivasan Subramaniyan, and Xiaorui Wang. Latency-guaranteed co-location of inference and training for reducing data center expenses. In *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*, 2024.

[55] Srinivasan Subramaniyan, Rudra Joshi, Xiaorui Wang, and Marco Brocanelli. Seeb-gpu: Early-exit aware scheduling and batching for edge gpu inference. In *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing*, SEC '25, New York, NY, USA, 2025. Association for Computing Machinery.

[56] Yuan Ma, Srinivasan Subramaniyan, and Xiaorui Wang. Power capping of gpu servers for machine learning inference optimization. In *Proceedings of the 54th International Conference on Parallel Processing (ICPP)*, 2025.

[57] K Vanishree, Anu George, Srivatsav Gunisetty, Srinivasan Subramanian, Shravan Kashyap, and Madhura Purnaprajna. Coin: Accelerated cnn co-inference through data partitioning on heterogeneous devices. In *2020 6th International Conference on Advanced Computing and Communication Systems (ICACCS)*, pages 90–95. IEEE, 2020.

[58] Qizhen Weng et al. Mlaas in the wild: Workload analysis and scheduling in large-scale heterogeneous gpu clusters. In *NSDI*, 2022.

[59] Jing Guo et al. Who limits the resource efficiency of my datacenter: An analysis of alibaba datacenter traces. In *IWQoS*, 2019.

[60] Feitelson et al. Packing schemes for gang scheduling. In *workshop on job scheduling strategies for parallel processing*, 1996.

[61] Zhenhua Han, Haisheng Tan, Shaofeng H-C Jiang, Xiaoming Fu, Wanli Cao, and Francis CM Lau. Scheduling placement-sensitive bsp jobs with inaccurate execution time estimation. In *IEEE INFOCOM 2020-IEEE Conference on Computer Communications*, 2020.

[62] Wencong Xiao et al. Gandiva: Introspective cluster scheduling for deep learning. In *OSDI*, 2018.

[63] Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020.

[64] Deepak Narayanan et al. Analysis and exploitation of dynamic pricing in the public cloud for ml training. In *VLDB DISPA Workshop*, 2020.

[65] Kuangyu Zheng et al. Joint power optimization of data center network and servers with correlation analysis. In *INFOCOM*, 2014.

[66] Stuart Mitchell, Michael OSullivan, and Iain Dunning. Pulp: a linear programming toolkit for python. *The University of Auckland, Auckland, New Zealand*, 2011.

[67] NVIDIA Corporation. *NVIDIA System Management Interface*, 2023.

[68] Paul Delestrac, Debjyoti Battacharjee, Simei Yang, Diksha Moolchandani, Francky Catthoor, Lionel Torres, and David Novo. Multi-level analysis of gpu utilization in ml training workloads. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2024.

[69] Gao Huang et al. Densely connected convolutional networks. In *CVPR*, 2017.

[70] Christian Szegedy, Sergey Ioffe, Vincent Vanhoucke, and Alexander Alemi. Inception-v4, inception-resnet and the impact of residual connections on learning. In *AAAI*, 2017.

[71] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.

[72] Christian Szegedy et al. Going deeper with convolutions. In *CVPR*, 2015.

[73] Karen Simonyan et al. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

[74] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, 2021.

[75] Gingfung Yeung, Damian Borowiec, Adrian Friday, Richard Harper, and Peter Garraghan. Towards gpu utilization prediction for cloud deep learning. In *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 20)*, 2020.

[76] Wei Gao, Zhisheng Ye, Peng Sun, Yonggang Wen, and Tianwei Zhang. Chronus: A novel deadline-aware scheduler for deep learning training jobs. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2021.

[77] Yi Zhu et al. Densenet for dense flow. In *ICIP*.

[78] Mingxing Tan et al. Mnasnet: Platform-aware neural architecture search for mobile. In *IEEE CVPR*, 2019.

[79] Scott Boag, Parijat Dube, Benjamin Herta, Waldemar Hummer, Vatche Ishakian, K Jayaram, Michael Kalantar, Vinod Muthusamy, Priya Nagpurkar, and Florian Rosenberg. Scalable multi-framework multi-tenant lifecycle management of deep learning training jobs. In *Workshop on ML Systems, NIPS*, 2017.

[80] Robert Grandl, Ganesh Ananthanarayanan, Srikanth Kandula, Sriram Rao, and Aditya Akella. Multi-resource packing for cluster schedulers. *ACM SIGCOMM Computer Communication Review*, 44(4), 2014.

[81] Xiaorui Wang and Ming Chen. Cluster-level feedback power control for performance optimization. In *IEEE 14th International Symposium on High Performance Computer Architecture (HPCA)*, 2008.

[82] Xiaorui Wang, Ming Chen, Charles Lefurgy, and Tom W Keller. Ship: Scalable hierarchical power control for large-scale data centers. In *18th International Conference on Parallel Architectures and Compilation Techniques*, 2009.

[83] Charles Lefurgy, Xiaorui Wang, and Malcolm Ware. Server-level power control. In *IEEE Fourth International Conference on Autonomic Computing (ICAC'07)*, 2007.

[84] Robert A Bridges, Neena Imam, and Tiffany M Mintz. Understanding gpu power: A survey of profiling, modeling, and simulation methods. *ACM Computing Surveys (CSUR)*, 49(3), 2016.

[85] Wenjie Liu, Zhihui Du, Yu Xiao, David A Bader, and Chen Xu. A waterfall model to achieve energy efficient tasks mapping for large scale gpu clusters. In *IEEE International Symposium on Parallel and Distributed Processing Workshops and Phd Forum*, 2011.

[86] Kai Ma, Xue Li, Wei Chen, Chi Zhang, and Xiaorui Wang. Greengpu: A holistic approach to energy efficiency in gpu-cpu heterogeneous architectures. In *41st international conference on parallel processing (ICPP)*. IEEE, 2012.

[87] Federica Filippini, Danilo Ardagna, Marco Lattuada, Edoardo Amaldi, Maciek Riedl, Katarzyna Materka, Paweł Skrzypek, Michele Ciavotta, Fabrizio Magugliani, and Marco Cicala. Andreas: Artificial intelligence training scheduler for accelerated resource clusters. In *2021 8th International Conference on Future Internet of Things and Cloud (FiCloud)*. IEEE, 2021.

[88] Simon Wu and Xiaorui Wang. Exploiting hybrid energy storage to minimize the carbon footprint of AI data centers. In *Proceedings of the 40th ACM International Conference on Supercomputing*, ICS '26, Belfast, Northern Ireland, United Kingdom, July 2026. Association for Computing Machinery.

[89] Simon Wu, Yuan Ma, and Xiaorui Wang. Phase-aware peak power reduction for minimizing the capital expense of LLM inference. In *Proceedings of the 40th ACM International Conference on Supercomputing*, ICS '26, Belfast, Northern Ireland, United Kingdom, July 2026. Association for Computing Machinery.

[90] Diandian Gu, Xintong Xie, Gang Huang, Xin Jin, and Xuanzhe Liu. Energy-efficient gpu clusters scheduling for deep learning. *arXiv preprint arXiv:2304.06381*, 2023.

[91] Zhihao Jia, Matei Zaharia, and Alex Aiken. Beyond data and model parallelism for deep neural networks. *Proceedings of Machine Learning and Systems*, 2019.

[92] Arpan Jain, Ammar Ahmad Awan, Asmaa M Aljuhani, Jahanzeb Maqbool Hashmi, Quentin G Anthony, Hari Subramoni, Dhableswar K Panda, Raghu Machiraju, and Anil Parwani. Gems: Gpu-enabled memory-aware model-parallelism system for distributed dnn training. In *SC20: international conference for high performance computing, networking, storage and analysis*, 2020.

[93] Pierre Jacquet, Maxime Agusti, Eddy Caron, Camille Coti, Marcos Dias De Assunção, Laurent Lefèvre, and Anne-Cécile Orgerie. Untangling gpu power consumption: Job-level inference in cloud shared settings. In *European Conference on Computer Systems (Eurosys)*, 2026.

[94] Yanjie Gao, Yu Liu, Hongyu Zhang, Zhengxian Li, Yonghao Zhu, Haoxiang Lin, and Mao Yang. Estimating gpu memory consumption of deep learning models.

In *Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2020.

[95] Adrian Osterwind, Julian Droste-Rehling, Manoj-Rohit Vemparala, and Domenik Helms. Hardware execution time prediction for neural network layers. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2022.

[96] Xiaoming Chen, Danny Z Chen, and Xiaobo Sharon Hu. modnn: Memory optimal dnn training on gpus. In *IEEE Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2018.

[97] Hadeel Albahar, Shruti Dongare, Yanlin Du, Nannan Zhao, Arnab K Paul, and Ali R Butt. Schedtune: A heterogeneity-aware gpu scheduler for deep learning. In *22nd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 2022.

[98] Seungbeom Choi, Sunho Lee, Yeonjae Kim, Jongse Park, Youngjin Kwon, and Jaehyuk Huh. Serving heterogeneous machine learning models on multi-gpu servers with spatio-temporal sharing. In *2022 USENIX Annual Technical Conference (USENIX ATC)*, 2022.

[99] Wenyan Chen, Zizhao Mo, Huanle Xu, Kejiang Ye, and Chengzhong Xu. Interference-aware multiplexing for deep learning in gpu clusters: A middleware approach. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.

[100] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, fine-grained gpu sharing for ml applications. In *Proceedings of the Nineteenth European Conference on Computer Systems*, 2024.

[101] Ties Robroek, Ehsan Yousefzadeh-Asl-Miandoab, and Pınar Tözün. An analysis of collocation on gpus for deep learning training. In *Proceedings of the 4th Workshop on Machine Learning and Systems*, 2024.

[102] Paul Elvinger, Foteini Strati, Natalie Enright Jerger, and Ana Klimovic. Measuring gpu utilization one level deeper. *arXiv preprint arXiv:2501.16909*, 2025.

[103] Srinivasan Subramaniyan and Xiaorui Wang. Fc-gpu: Feedback control gpu scheduling for real-time embedded systems. 24(5s), September 2025.

[104] Sheng Zhang, Zhuzhong Qian, Zhaoyi Luo, Jie Wu, and Sanglu Lu. Burstiness-aware resource reservation for server consolidation in computing clouds. *IEEE Transactions on Parallel and Distributed Systems*, 27(4), 2015.

[105] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. Cassini:network-aware job scheduling in machine learning clusters. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.

[106] Guin Gilman and Robert J Walls. Characterizing concurrency mechanisms for nvidia gpus under deep learning workloads. *ACM SIGMETRICS Performance Evaluation Review*, 49(3), 2022.

[107] Mu Li, David G. Andersen, Jun Woo Park, Alexander J. Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J. Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2014.

[108] Sanghyun Woo, Shoubhik Debnath, Ronghang Hu, Xinlei Chen, Zhuang Liu, In So Kweon, and Saining Xie. Convnext v2: Co-designing and scaling convnets with masked autoencoders. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023.

[109] Forrest N Iandola, Song Han, Matthew W Moskewicz, Khalid Ashraf, William J Dally, and Kurt Keutzer. Squeezenet: Alexnet-level accuracy with 50x fewer parameters and¡ 0.5 mb model size. *arXiv preprint arXiv:1602.07360*, 2016.

[110] Jayashree Mohan, Amar Phanishayee, Janardhan Kulkarni, and Vijay Chidambaram. Looking beyond gpus for dnn scheduling on multi-tenant clusters. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.